



Báo cáo tóm tắt
Một số thuật toán tiêu biểu trong SPMF

Tên sinh viên: Lê Thị Thanh Vân

MSSV: 030239230288

Lớp sinh hoạt: DH39KH02

MỤC LỤC

I. Frequent Itemset mining	3
1. <i>Maximal, Closed and Generator itemsets</i>	4
a. <i>Vấn đề của frequent itemsets</i>	4
b. <i>Vấn đề của phương pháp tăng minsup</i>	5
c. <i>Phương pháp giải quyết</i>	5
2. <i>Rare Itemset Mining and the AprioriInverse and AprioriRare algorithms</i>	10
a. <i>Vấn đề</i>	10
b. <i>Giải pháp</i>	10
□ <i>Minimal rare itemset: dùng Thuật toán APrioriRare để tìm :</i>	10
Cách thuật toán hoạt động:	10
□ <i>Perfectly rare itemsets: dùng Thuật toán AprioriInverse để tìm:</i>	12
3. <i>Correlated and statistically significant itemsets, including the CORI algorithm</i>	13
II. High-Utility Pattern Mining	14
1. <i>High Utility Itemset Mining</i>	14
2. <i>Minimal High-Utility Itemsets (the MinFHM algorithm)</i>	16
3. <i>Mining Correlated High Utility Patterns (the FCHM algorithm)</i>	16
4. <i>TKQ:</i>	17
5. <i>Cross-level high utility itemset mining</i>	17
III. Episode Mining	18
a. <i>Một số định nghĩa cơ bản:</i>	18
b. <i>Một số thuật toán cơ bản :</i>	18
IV. Sequential Pattern Mining	20
a. <i>Mục đích:</i>	20
b. <i>Một số định nghĩa, phép tính cơ bản cần nắm:</i>	20
c. <i>Một số thuật toán cơ bản:</i>	20
VI. Sequence Prediction (source code version only)	23
a. <i>Mục tiêu:</i>	24
b. <i>Ứng dụng: Ứng dụng rộng rãi trong thực tế như:</i>	24
c. <i>Các Phương Pháp Truyền Thống & Hạn Chế</i>	24
d. <i>Tổng kết hạn chế của các mô hình cũ:</i>	26
e. <i>Giải Pháp: Mô Hình CPT (Compact Prediction Tree)</i>	26
f. <i>CPT+ (Compact Prediction Tree Plus) - Tối ưu CPT</i>	27
II. Periodic pattern mining	27
1. <i>Luật chu kỳ (Period rules)</i>	27

2.	<i>Các loại mẫu phân loại theo chiều thời gian (time dimension)</i>	27
3.	<i>Khai thác tập mục có chu kỳ (Periodic Itemset Mining)</i>	28
VII.	Time Series Mining	29
1.	Định nghĩa:	30
2.	<i>Quy trình xử lý được đề xuất theo một logic tuần tự hợp lý:</i>	30
2.1	<i>Trực quan hóa và Làm mịn (Visualization & Smoothing)</i>	30
2.2.	<i>Biến đổi và Rút trích Đặc trưng (Transformation & Feature Extraction)</i>	32
2.3.	<i>Phân đoạn và Biểu diễn Ký hiệu (Segmentation & Symbolic Representation)</i>	34

I. Frequent Itemset mining

- Đầu tiên là khai thác tập mục phổ biến. trong những mặt hàng được bán ở cửa hàng thì việc xác định món hàng nào là thường xuyên được mua sẽ hữu ích, và việc xác định này đóng góp vào việc tìm ra những các set hàng hay có thể đáp ứng được nhu cầu của khách hàng tốt hơn (và những set hàng này được đề xuất ra từ những thuật toán với dữ liệu đầu vào là những lịch sử hành vi giao dịch của khách hàng). Từ việc hiểu những thuật toán đơn giản như apriori (thật ra thuật toán này không phải là tốt nhất nhưng nó nổi tiếng và làm nền tảng cho sự phát triển các thuật toán khác về sau), cụ thể như cách tính support của một món hàng hay một luật kết hợp, confidence và lift của luật kết hợp cùng với những biến thể của nó như Apriori_TID và Apriori_Bitset (những biến thể như vậy như là một sự tối ưu hoá khi tiếp cận với dữ liệu lớn ngoài thực tế - như nên lưu dữ liệu hay chuyển hoá, sắp xếp nó ra sao để khi scan dữ liệu để khi phân tích ít tốn tài nguyên như thời gian chạy, bộ nhớ. Mặc dù đồng ý rằng hiệu suất của thuật toán còn phụ thuộc vào đặc tính của dữ liệu đầu vào là real data, fake data, dữ liệu phân bố dày đặc hay thưa,... Một số cách tối ưu : thay vì lưu bằng chữ thì quy định nó về dạng số, khi quét dữ liệu thì quét từ trên xuống nên mình sẽ sắp xếp các giao dịch theo tăng dần độ dài các mặt hàng được mua, hay là chuyển sang TID_list để dùng intersection (phép giao), Logical_And để tính support, và có một thuật ngữ 'total order' - là cách mình quy định cho dữ liệu được lưu và có thể theo bảng chữ cái nhưng tốt nhất là theo thứ tự support of a single item tăng dần // cây deep - search // , hay như ECLAT khi lưu trữ thì chia làm lưu tiền tố và hậu tố thay vì lưu một cách truyền thống giúp tiết kiệm bộ nhớ.

Optimization 3 - memory

Consider an equivalence class:
 $\{ABCD, ABCE, ABCF, ABCG, ABCH\}$

It can be stored more efficiently as:
 $P = \{ABC\}$
 $E = \{E, F, G, H\}$

33:36 / 37:14

Đi sâu hơn nữa, 2 thuộc tính mà Apriori dựa vào để hoạt động, 1 trong đó là tính không đơn điệu của support, tất cả đều giúp giảm không gian tìm kiếm đáng kể rất quan trọng khi làm việc với khối lượng lớn dữ liệu.

Nhìn chung, ở một số thuật toán, em hiểu cần input những gì, và sau đó từng bước hoạt động của thuật toán để có được output. Và vấn đề có thể phát sinh hầu hết là ở input hoặc trong quá trình xử lý dữ liệu và từ việc khắc phục đó sẽ dẫn đến những thuật toán mới. Như minsup, khi tăng lên cao thì giảm không gian tìm kiếm và tốc độ nhanh hơn nhưng có thể bỏ qua những mặt hàng hiếm nhưng lại mang lại doanh thu đáng kể. Hay trong quá trình thuật toán Apriori hoạt động, lúc đầu là quét dữ liệu để tính support cho một món hàng đơn, sau đó loại những món không thỏa ngưỡng tối thiểu, và kết hợp những món đơn thành đôi và cứ làm như thế. Nhưng khi kết hợp sẽ có một số vấn đề kỹ thuật như

$\{A, B\}$ and $\{A, E\} \rightarrow \{A, B, E\}$

$\{B, E\}$ and $\{A, E\} \rightarrow \{A, B, E\}$

=> problem: some candidates are generated several times!

Khai phá luật kết hợp: khám phá những sản phẩm mà khách hàng thích mua cùng với nhau .

Giải thích : Sau khi đã khai phá được frequent itemsets, ví dụ { pasta, cake } thì frequent, có một câu hỏi đặt ra là chúng ta liệu có kết luận rằng người mua pasta thì sẽ mua cake ? Thì việc tìm hiểu “Khai phá luật kết hợp sẽ là câu trả lời”. Khác với khai phá tập mục phổ biến, ở đây sẽ học cách tính support của luật, và thêm các thông số mới như confidence và lift. Support luật kết hợp cho biết tỉ lệ của set hàng đó so với tổng số giao dịch, còn confidence như một xác suất có điều kiện (Nếu A thì B) thì mức độ tin cậy là bao nhiêu %. Nhưng confidence sẽ dễ dẫn đến nhầm lẫn khi đo lường định tính (sở thích)

Another problem

- Consider: {tea} → {coffee} 
support : 15% confidence : 75 %
- This seems like an interesting pattern...

	Coffee	$\overline{\text{Coffee}}$	
Tea	150	50	200
$\overline{\text{Tea}}$	650	150	800
	800	200	1000

$$\text{conf}(X \rightarrow Y) = \frac{\text{sup}(XY)}{\text{sup}(X)}$$

- However, 80 % of the people drink coffee no matter if they drink tea or not.
- In fact, the probability of drinking coffee for tea drinkers is lower than for non tea drinkers (80 instead of 75 %)!
- This problem occurs because the confidence measure does not consider the support of the left side of rules.

dù độ tin cậy 75% nhưng việc không uống trà dẫn đến uống cà phê nó chiếm ưu thế hơn. Xuất phát từ công thức conf không nhắc đến sup(Y) nên từ đó lift ra đời để nói lên mối tương quan giữa hai biến X , Y.

Một số phép đo lường khác: Jaccard - chữ ký, cosine..

1. Maximal, Closed and Generator itemsets

a. Vấn đề của frequent itemsets

Nối tiếp phần frequent itemsets, như cách hiểu trên lý thuyết , chỉ cần thỏa mãn ngưỡng minsup, minconf thì những tập mục sẽ được coi là phổ biến. Nhưng trong thực tế, khi trực tiếp xử lý dữ liệu lớn , sẽ gặp 2 vấn đề chính, một là số lượng của các tập phổ biến rất lớn , hai là ‘các hình thức dư’ (VD: {A, B, C} ; {B,C}). Mục đích ban đầu khi dùng thuật toán là để tìm ra dữ liệu mà nó ‘interesting’ , vậy thì việc dư thừa đặc biệt với một số lượng lớn như vậy sẽ giảm hiệu quả thuật toán và không có đóng góp gì vào công việc kinh doanh, làm nhiễu loạn thông tin và từ đó có thể gây ra việc hiểu sai bản chất vấn đề , tổn chi phí tài nguyên . Việc thấu hiểu vấn đề như vậy là bước tiến cho sự ra đời các thuật toán **concise representation of frequent itemsets** , nghĩa là không chỉ phổ biến mà nó còn mang tính đại diện chính xác hơn, như thay vì output {A, B, C} và {B, C} thì làm cách nào để tóm gọn, tóm tắt frequent itemsets.

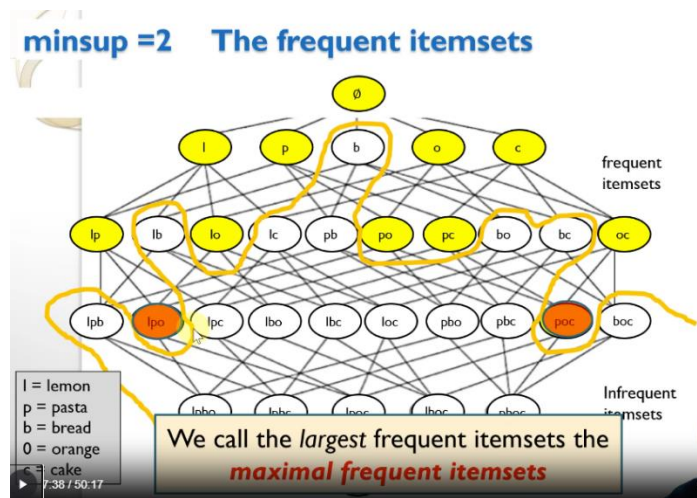
b. Vấn đề của phương pháp tăng minsup

Trong môn học Big Data, em đã từng tối ưu thuật toán bằng một phương pháp là tăng support vì dễ làm, nhanh chóng và giảm được số lượng đầu ra ngay lập tức. Nhưng nhược lại lớn hơn ưu, vì bản chất nó chỉ đơn thuần là bỏ bớt đi số lượng chứ không thể cắt bỏ phần dư và để lại phần đại diện. Và phương pháp thô này cũng rất nguy hiểm cho phân tích kinh doanh. Một nguy cơ khi cắt giảm chỉ dựa số lượng và không dựa vào bản chất hiểu dữ liệu sẽ dẫn đến việc làm mất đi các insight quý giá (một số itemsets có support thấp nhưng lại có lợi nhuận cao - vấn đề này được đề cập ở phía sau). Nên khi tăng support, thực sự là một phương pháp giảm số lượng output chứ chưa 'làm sạch' thông tin.

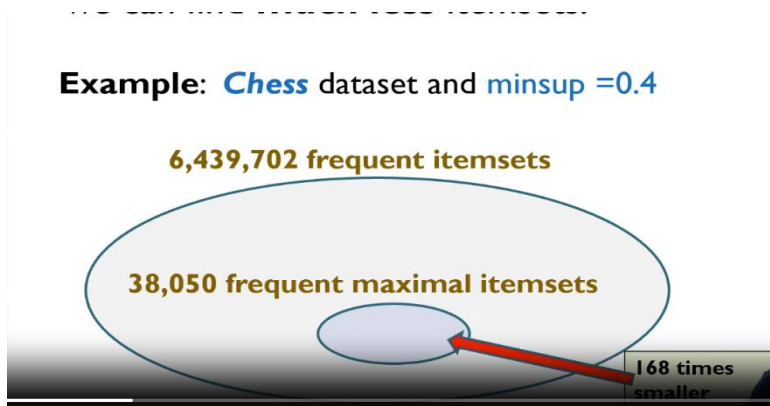
c. Phương pháp giải quyết

Concise representation (Cách tiếp cận tinh tế) thay vì loại bỏ dữ liệu, thì thông tin sẽ được nén lại một cách thông minh. Bằng phương pháp này, có 2 cách tiếp cận chính là tập phổ biến cực đại (Maximal Frequent Itemsets) và tập phổ biến đóng (Closed Frequent Itemsets).

Về tập phổ biến cực đại (Maximal Frequent Itemsets):



- Như vậy ta có thể tiết kiệm được không gian lưu trữ, có tính nhỏ gọn (compact) cũng như tiết kiệm thời gian chạy thuật toán. Nhưng có một nhược điểm là bị mất đi dữ liệu về support.



Một số thuật toán giúp tìm maximal itemsets ?

GenMax: a modified version of ECLAT

FPMMax: a modified version of FP-Growth

LCMMax: a modified version of LCM

Về tập phổ biến đóng (Closed Frequent Itemsets):

Definition: A (frequent) itemset X is closed if there exists no (frequent) itemset Y such that $X \subset Y$ and $\text{sup}(X) = \text{sup}(Y)$.

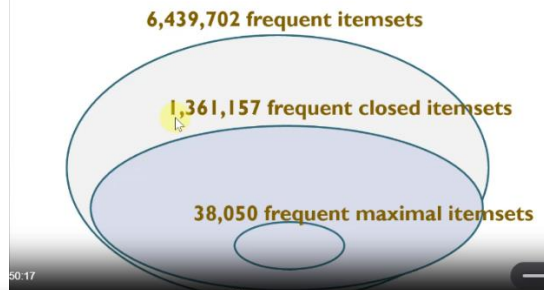
Closed Frequent Itemsets khắc phục được nhược điểm của Maximal Frequent Itemsets là vẫn giữ được số support.

Closed itemsets

{pasta}	support = 4	closed
{lemon}	support = 3	
{orange}	support = 3	
{cake}	support = 2	
{pasta, lemon}	support: 3	closed
{pasta, orange}	support: 3	closed
{pasta, cake}	support: 2	
{lemon, orange}	support: 2	
{orange, cake}	support: 2	
{pasta, lemon, orange}	support: 2	closed
{pasta, orange, cake}	support: 2	closed

Closed itemsets are compact

Example: Chess dataset and minsup = 0.4



Một số thuật toán để tìm the closed itemsets:

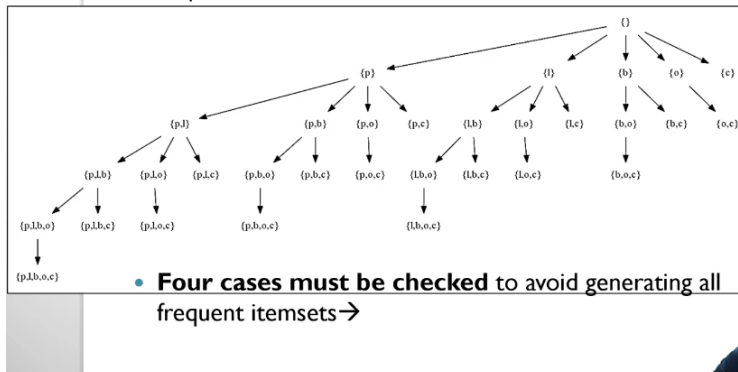
DCI_Closed, FPClosed, Closet, LCM, ..

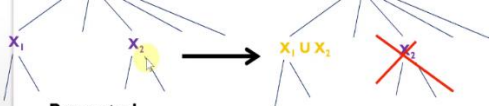
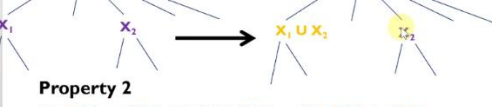
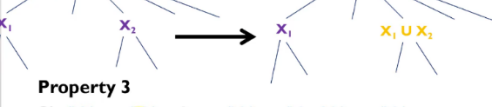
Và để tìm closed itemsets mà tránh được việc generate all frequent itemset, thuật toán Charm là một giải pháp.

Charm algorithm

Charm (Zaki, 2001)

- Based on ECLAT
- Depth-first search



Thuộc tính 1: Nếu $t(X1) = t(X2)$:	Four cases Suppose that the algorithm is considering combining an itemset X_1 with an itemset X_2 such that $X_2 > X_1$.  Property 1 IF $t(X_1) = t(X_2)$, THEN $t(X_1) = t(X_2) = t(X_1 \cup X_2)$. - Thus, we can replace all occurrences of X_1 by $X_1 \cup X_2$. - We do not need to consider X_2. Notation: $t(X)$ denotes the set of transactions containing an itemset X	Kết luận : $c(X1) = c(X2) = c(X1 \cup X2)$ Hành động : THAY THẾ + LOẠI BỎ Giải thích: Vì $t(X1) = t(X2)$, nghĩa là bất cứ giao dịch nào chứa $X1$ cũng chứa $X2$ và ngược lại. Do đó, closure của chúng bằng nhau. Việc thay thế $X1$ bằng $X1 \cup X2$ là để ‘kéo’ item từ $X2$ sang $X1$, tạo thành một itemset lớn hơn ngay lập tức mà có cùng closure. Việc loại bỏ $X2$ là vì nó sẽ không tạo ra một closure mới nào khác với $X1 \cup X2$. Giữ lại $X2$ chỉ dẫn đến việc tính toán dư thừa.
Thuộc tính 2: Nếu $t(X1)$ là con của $t(X2)$	Four cases Suppose that the algorithm is considering combining an itemset X_1 with an itemset X_2 such that $X_2 > X_1$.  Property 2 IF $t(X_1) \subset t(X_2)$, THEN $t(X_1) = t(X_1 \cap X_2) \neq t(X_2)$. - Thus, we can replace all occurrences of X_1 by $X_1 \cup X_2$. - But we must keep X_2.	Kết luận: $c(X1) = c(X1 \cup X2) \neq c(X2)$ Hành động: THAY THẾ + GIỮ LẠI Giải thích: Nếu ở thuộc tính 1 việc loại bỏ vì lý do dư thừa, còn ở thuộc tính 2, $X2$ có tồn tại trong một số giao dịch không chứa $X1$ nhưng có thể sẽ dẫn đến một closure khác với $X1$ cần được khám phá riêng. Nếu loại bỏ $X2$, thuật toán có thể bỏ sót một closed itemset tiềm năng. Nên thay vì loại bỏ, ta sẽ giữ lại $X2$.
Thuộc tính 3: Là đảo phía lại của thuộc tính 2, cách hiểu và làm vẫn tương tự	Four cases Suppose that the algorithm is considering combining an itemset X_1 with an itemset X_2 such that $X_2 > X_1$.  Property 3 Si $t(X_1) \supset t(X_2)$, alors $t(X_2) = t(X_1 \cap X_2) \neq t(X_1)$. - Thus, we can replace all occurrences of X_2 by $X_1 \cup X_2$. - But we must keep X_1.	
Thuộc tính 4: Nếu $t(X1) \neq t(X2)$:	Four cases Suppose that the algorithm is considering combining an itemset X_1 with an itemset X_2 such that $X_2 > X_1$.  Property 4 IF $t(X_1) \neq t(X_2)$, then $t(X_2) \neq t(X_1 \cap X_2) \neq t(X_1)$. We cannot remove anything.	Kết luận: $c(X1) \neq c(X1 \cup X2) \neq c(X2)$ Hành động: GIỮ LẠI + Thêm lớp con mới $X1 \cup X2$ để khám phá đệ quy Giải thích: Cả $X1$ và $X2$ đều dẫn đến các closure khác nhau và đều có giá trị khám phá. Do đó, cả hai đều phải được giữ lại. Itemset $X1 \cup X2$ được tạo ra và thêm vào lớp con để xử lý ở level sâu hơn, vì nó có thể dẫn đến một closure thứ ba, khác với closure của $X1$ và $X2$

Dù đã áp dụng 4 properties, CHARM vẫn có thể sinh ra một số tập không đóng. Nó sử dụng một bảng băm (hash table) thông minh để kiểm tra nhanh xem một tập vừa tìm ra đã bị "bao hàm" (subsumed) bởi một tập đóng nào đó đã biết chưa. Nếu có, nó sẽ loại bỏ.

Tối ưu thuật toán Charm: ý tưởng diffset

Optimizations

- Implementation with diffsets (**dCharm**).
- Implementation with bit vectors.
- Use a **triangular matrix** to count the support of itemsets of size 2 by scanning the database once.
- ...

	E	D	C	B
A	0	0	0	0
B	0	0	0	
C	0	0		
D	0			

Diffsets: Thay vì lưu trữ cả một tập transaction ID (tidlist) lớn, CHARM chỉ lưu sự khác biệt (difference) so với node cha. Ví dụ, nếu node cha có tidlist {1,2,3,4,5} và node con có tidlist {1,3,5}, diffset sẽ là {2,4}. Cách này tiết kiệm bộ nhớ cực kỳ hiệu quả (theo bài báo, có thể giảm 50 lần), giúp xử lý các dataset lớn trong bộ nhớ.

So sánh Charm với Apriori

	Charm	Apriori
Output	Closed itemsets	Liệt kê tất cả frequent itemsets
Cách nhìn dữ liệu	Nhìn dọc (Vertical - IT - P Pairs): nhìn theo từng cột (item) : Mặt hàng A được mua trong các transaction {1,3,4,5}", "Mặt hàng C được mua trong {1,2,3,4,5,6}", ...	Nhìn dữ liệu theo từng dòng (transaction): "Transaction 1 mua {A, C, T, W}", "Transaction 2 mua {C, D, W}", ... Rất trực quan nhưng tốn kém khi tính toán kết hợp.

The generator itemsets (or key itemsets)

The generator itemsets (or key itemsets)

Definition: A (frequent) itemset X is a **generator** if there exists no (frequent) itemset Y such that $Y \subset X$ and $\text{sup}(X) = \text{sup}(Y)$.

In the example ($\text{minsup} = 2$), there are five **frequent generator itemsets**:

$\{\}$	support: 4
$\{\text{lemon}\}$	support: 3
$\{\text{orange}\}$	support: 3
$\{\text{cake}\}$	support: 2
$\{\text{lemon, orange}\}$	support: 2

Để ‘summarise frequent itemsets’, ngoài việc giảm kích thước tối đa đầu ra bằng việc chỉ giữ lại tập lớn nhất (Maximal Itemsets) hay nén thông tin mà không bị mất mát mà vẫn biểu diễn đầy đủ nhất (Closed Itemsets), thì còn một phương pháp nữa là tìm nguyên nhân tối thiểu, ‘mã khoá’ ngắn gọn nhất.

Ví dụ:

$\{\text{cake}\}$ là generator của $\{\text{orange, cake}\}$: Điều này cho thấy việc thêm orange vào $\{\text{cake}\}$ không làm thay đổi nhóm khách hàng (support vẫn là 2). Trong kinh doanh, điều này có thể được hiểu là những khách hàng mua cake trong cửa hàng này luôn luôn mua kèm orange? Không hẳn, nhưng trong dataset nhỏ này, nó cho thấy một xu hướng đáng chú ý.

$\{\text{lemon, orange}\}$ là generator: Điều này cho thấy không có mặt hàng đơn lẻ nào (lemon hay orange) có thể xác định duy nhất nhóm khách hàng mua cả hai mặt hàng này. Phải có cả hai thì mới "khóa" được nhóm khách hàng đó.

Tóm lại, generators giống như những "mã khóa" ngắn gọn nhất để mở ra các nhóm khách hàng cụ thể. Việc tìm ra chúng giúp các nhà kinh doanh tập trung vào các yếu tố tối thiểu nhưng mang tính quyết định trong hành vi mua hàng.

- Ứng dụng hay : Bài toán Phân Lớp (Classification):

Trong các thuật toán phân lớp dựa trên luật kết hợp, người ta cần chọn ra các luật tốt nhất để dự đoán nhãn lớp.

Các nghiên cứu thực nghiệm chỉ ra rằng các luật được sinh ra từ Generators thường cho độ chính xác phân lớp cao hơn so với luật từ Closed hay Maximal Itemsets.

Lý do: Generators là các mẫu hình (pattern) "tinh khiết" và "ổn định" hơn. Chúng tránh được hiện tượng overfitting (dễ xảy ra với các tập quá dài như Maximal) và vẫn giữ được tính đặc trưng (trong khi các tập quá ngắn thì lại quá phổ biến và không có tính phân biệt).

- Một số thuật toán tìm ra generator

Pascal, DefMe, ...v..

Find generator itemsets by avoiding generating all frequent itemsets

- Key search space pruning property:

An itemset can only be a generator if all its subsets are also generators.

So sánh Maximal, Closed, Generator Itemsets

Tiêu chí	Generator Itemsets (Tập Sinh)	Closed Itemsets (Tập Đóng)	Maximal Itemsets (Tập Cực Đại)
Định nghĩa	Tập nhỏ nhất mà không có tập con nào có cùng support Tập rỗng cũng là generator	Tập lớn nhất mà không có tập cha nào có cùng support	Tập lớn nhất mà không có tập cha nào là frequent. Chỉ quan tâm đến kích thước, không quan tâm support.
Mục tiêu	Tìm nguyên nhân tối thiểu, ‘mã khoá’ ngắn gọn nhất	Nén thông tin mà không mất mát. Biểu diễn đầy đủ nhất	Giảm kích thước tối đa đầu ra. Chỉ giữ lại các tập lớn nhất.
Tính chất	Tối giản (Minimal)	Đầy đủ (Complete)	Cực đại (Maximal)
Ứng dụng	Sinh luật kết hợp mạnh Tạo đặc trưng cho phân lớp	Tổng quan dữ liệu. Suy ngược support của các tập	Phân tích nhanh, Xác định các xu hướng lớn nhất.
Support	Giữ lại	Giữ lại (và có thể suy ngược)	Mất mát hoàn toàn

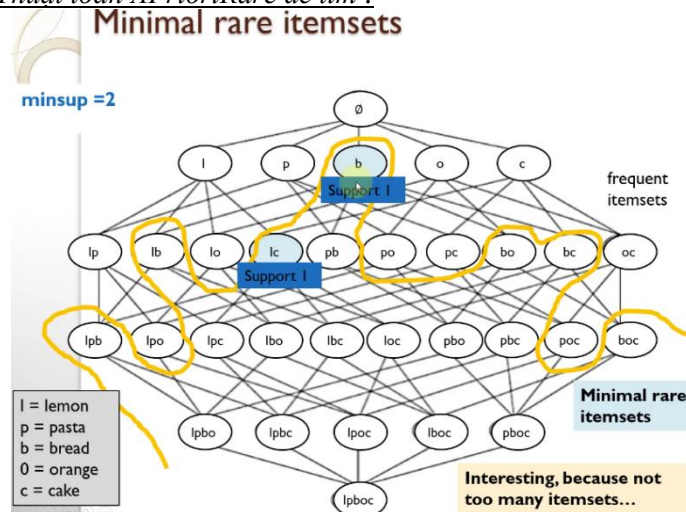
2. Rare Itemset Mining and the AprioriInverse and AprioriRare algorithms

a. Vấn đề

- Vẫn là vấn đề những tập mục phổ biến không phải lúc nào nó cũng mang insights hay để khai phá vì thực tế có những tập mục biến dư thừa. Để khắc phục vấn đề này, người ta tạo ra những thuật toán để ‘summarise frequent itemsets’. Ngoài ra, còn có một cách làm khác đó là chuyển hướng sang khám phá từ những tập mục hiếm (không thoả minsup).
- Tập mục hiếm: là những tập mục không thoả minsup
- Vấn đề của tập mục hiếm: support = 0. Khi support = 0, không hữu ích để đi tìm, mà mỗi khi làm việc với dữ liệu, cần phải có dữ liệu để tìm kiếm, làm như vậy hàng triệu lần nếu trong dữ liệu lớn ngoài thực tế => not interesting !!!!

b. Giải pháp

⇒ Minimal rare itemset: dùng Thuật toán APrioriRare để tìm :



Cách thuật toán hoạt động:

The AprioriRare algorithm (2007)

- It is based on Apriori
- As Apriori, the itemsets are generated by levels:
 - Itemsets of size 1 (one item)
 - Itemsets of size 2 (two items)
 - Itemsets of size 3 (three items)
 - ...
- Two differences:
 - If AprioriRare finds an itemset of size k that is infrequent, AprioriRare checks if its subsets are frequent. If yes, it is a minimal rare itemset.
 - AprioriRare do not use the infrequent itemsets to generate larger itemsets.

The AprioriRare algorithm

Step 1: Scan the database to calculate the support of all itemsets of size 1.

e.g.

{pasta}	support = 4
{lemon}	support = 3
{bread}	support = 1
{orange}	support = 3
{cake}	support = 2

The AprioriRare algorithm

Step 2: Check all infrequent itemsets. If all subsets are frequent, they are minimal rare itemsets.

e.g.

{pasta}	support = 4
{lemon}	support = 3
{bread}	support = 1
{orange}	support = 3
{cake}	support = 2

The AprioriRare algorithm

Step 2: Check all infrequent itemsets. If all subsets are frequent, they are minimal rare itemsets.

e.g.

{pasta}	support = 4
{lemon}	support = 3
{bread}	support = 1
{orange}	support = 3
{cake}	support = 2

Minimal Rare Itemset

The AprioriRare algorithm

Step 2: Keep frequent itemsets.

e.g.

{pasta}	support = 4
{lemon}	support = 3
{orange}	support = 3
{cake}	support = 2

The AprioriRare algorithm

Step 3: Generate candidates of size 2 by combining pairs of frequent itemsets of size 1.

Candidates of size 2	
{pasta, lemon}	
{pasta, orange}	
{pasta, cake}	
{lemon, orange}	
{lemon, cake}	
{orange, cake}	

Frequent items

{pasta}
{lemon}
{orange}
{cake}

The AprioriRare algorithm

Step 5: Scan the database to calculate the support of remaining candidate itemsets of size 2.

Candidates of size 2	
{pasta, lemon}	support: 3
{pasta, orange}	support: 3
{pasta, cake}	support: 2
{lemon, orange}	support: 2
{lemon, cake}	support: 1
{orange, cake}	support: 2

The AprioriRare algorithm

Step 6: For each infrequent itemset, check if all the subsets are frequent

Candidates of size 2	
{pasta, lemon}	support: 3
{pasta, orange}	support: 3
{pasta, cake}	support: 2
{lemon, orange}	support: 2
{lemon, cake}	support: 1
{orange, cake}	support: 2

The AprioriRare algorithm Step 6: For each infrequent itemset, check if all the subsets are frequent Candidates of size 2 {pasta, lemon} support: 3 {pasta, orange} support: 3 {pasta, cake} support: 2 {lemon, orange} support: 2 {lemon, cake} support: 1 ← Minimal Rare Itemset {orange, cake} support: 2	The AprioriRare algorithm Step 6: Keep frequent itemsets of size 2 Frequent itemsets of size 2 {pasta, lemon} support: 3 {pasta, orange} support: 3 {pasta, cake} support: 2 {lemon, orange} support: 2 {orange, cake} support: 2
The AprioriRare algorithm Step 8: For each infrequent itemset, check if all the subsets are frequent Frequent itemsets of size 2 {pasta, lemon} {pasta, orange} {pasta, cake} {lemon, orange} {orange, cake} Candidates of size 3 {pasta, lemon, orange} {pasta, lemon, cake} {pasta, orange, cake} {lemon, orange, cake}	The AprioriRare algorithm Step 8: For each infrequent itemset, check if all the subsets are frequent Frequent itemsets of size 2 {pasta, lemon} {pasta, orange} {pasta, cake} {lemon, orange} {orange, cake} Candidates of size 3 {pasta, lemon, orange} {pasta, lemon, cake} {pasta, orange, cake} {lemon, orange, cake}
The AprioriRare algorithm Step 8: For each infrequent itemset, check if all the subsets are frequent Frequent itemsets of size 2 {pasta, lemon} {pasta, orange} {pasta, cake} {lemon, orange} {orange, cake} Candidates of size 3 {pasta, lemon, orange} {pasta, lemon, cake} {pasta, orange, cake} {lemon, orange, cake}	The AprioriRare algorithm Step 10: Keep the frequent itemsets (all) frequent itemsets of size 3 {pasta, lemon, orange} support: 2 {pasta, orange, cake} support: 2
The AprioriRare algorithm Step 11: generate candidates of size 4 by combining pairs c frequent itemsets of size 3. Frequent itemsets of size 3 {pasta, lemon, orange} {pasta, orange, cake} Candidates of size 4 {pasta, lemon, orange, cake}	The AprioriRare algorithm Step 12: Check to see if the subsets of each infrequent itemset are frequent. They are not. Frequent itemsets of size 3 {pasta, lemon, orange} {pasta, orange, cake} Candidates of size 4 {pasta, lemon, orange, cake}

⇒ Perfectly rare itemsets: dùng Thuật toán AprioriInverse để tìm:

Another definition: perfectly rare itemsets

Proposed for the **AprioriInverse** algorithm:

Yun Sing Koh, Nathan Rountree: *Finding Sporadic Rules Using AprioriInverse*. PAKDD 2005: 97-106

Definition 1 (perfectly rare itemset):

Let there be two thresholds minsup and maxsup , such that $\text{maxsup} > \text{minsup}$.

An itemset Z is a **frequent itemset** if $\text{sup}(Z) \geq \text{maxsup}$.

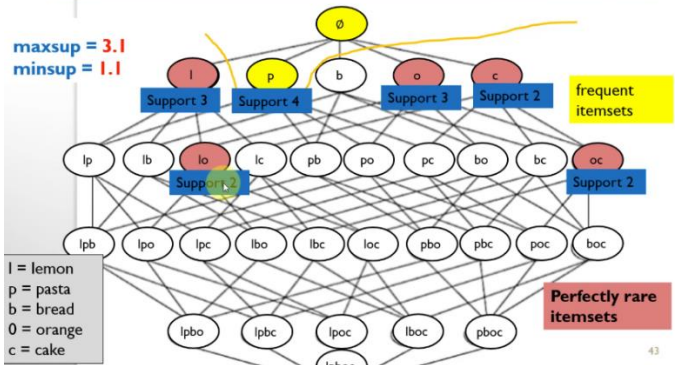
An itemset X is a **perfectly rare itemset** if $\text{sup}(X) \geq \text{minsup}$ and $\text{sup}(X) < \text{maxsup}$ and for any non empty subset $Y \subset X$, $\text{sup}(Y) \leq \text{maxsup}$.

Definition 1 (perfectly rare itemset):

Let there be two thresholds minsup and maxsup , such that $\text{maxsup} > \text{minsup}$.

An itemset Z is a **frequent itemset** if $\text{sup}(Z) \geq \text{maxsup}$.

An itemset X is a **perfectly rare itemset** if $\text{sup}(X) \geq \text{minsup}$, $\text{sup}(X) < \text{maxsup}$ and for any non empty subset $Y \subset X$, $\text{sup}(Y) \leq \text{maxsup}$.



maxsup trong AprioriInverse: Nó được dùng để loại bỏ các items quá phổ biến ngay từ đầu, để tập trung tìm kiếm các itemsets "hiếm nhưng có giá trị"

How to find perfectly rare itemsets?

- AprioriInverse (2005)
- Based on Apriori.
- Key difference:
 - Initially, AprioriInverse discards each item x such that $\text{sup}(x) > \text{maxsup}$ because we don't need such item.
 - After that AprioriInverse search for itemsets using the remaining items just like Apriori to find itemsets with a support no less than minsup .

3. Correlated and statistically significant itemsets, including the CORI algorithm

Đánh Giá và Khai Phá Các Tập Mục Tương Quan Mạnh

Một thách thức quan trọng trong khai phá tập phổ biến (frequent itemset mining) là nhiều tập mục được phát hiện có thể là "spurious" – nghĩa là chúng xuất hiện thường xuyên một cách tình cờ chứ không phải do mối quan hệ nội tại mạnh mẽ giữa các phần tử. Ví dụ, xét tập {pasta, cake} với độ phổ biến (support) 50% trong cơ sở dữ liệu giao dịch. Mặc dù có tần suất cao, tập này có thể không mang nhiều ý nghĩa vì pasta xuất hiện trong tất cả các giao dịch, khiến sự kết hợp trở nên yếu về mặt tương quan.

Để giải quyết vấn đề này và lọc ra những tập mục thực sự có ý nghĩa, ba hướng tiếp cận chính được đề xuất:

1. **Đánh giá tương quan bằng các hàm đo lường:** Các độ đo như **Bond** và **All-confidence** được sử dụng để định lượng sức mạnh của mối liên hệ giữa các item trong một tập.
 - **Bond** của một tập X được định nghĩa là tỷ lệ giữa support liên kết (conjunctive support) và support rời rạc (disjunctive support) của nó ($\text{bond}(X) = \text{sup}(X) / \text{disup}(X)$). Giá trị này nằm trong khoảng $[0, 1]$ và có tính chất anti-monotonic, cho phép tìm kiếm một cách hiệu quả. Thuật toán **CORI** là một ví dụ điển hình, kết hợp Eclat với tính toán Bond để khai phá các tập mục tương quan cao.

- **All-confidence** của một tập X là tỷ lệ giữa support của X và support lớn nhất của bất kỳ item đơn lẻ nào trong X ($\text{allconf}(X) = \text{sup}(X) / \max(\text{sup}(i) \forall i \in X)$). Cả Bond và All-confidence đều có tính chất **null-invariant** (bất biến với giao dịch không chứa tập mục), giúp chúng ổn định trước nhiễu. Có thể chứng minh rằng $\text{bond}(X) \leq \text{allconf}(X)$ với mọi tập X .
2. **Kiểm định ý nghĩa thống kê:** Các độ đo tương quan có thể không đảm bảo tính **ý nghĩa thống kê**. Việc áp dụng các kiểm định như **Kiểm định chính xác Fisher (Fisher Exact Test)** giúp xác định xem liệu mối tương quan quan sát được có khả năng cao là thực sự hay chỉ là ngẫu nhiên. Phương pháp này đặc biệt hữu ích trong các lĩnh vực như phân tích dữ liệu y tế.
 3. **Khai phá các mẫu tin khác:** Thay vì chỉ tập trung vào các tập mục, việc tìm kiếm **luật kết hợp (association rules)** hoặc các loại mẫu tin đặc thù khác như **tập mục không dư thừa (non-redundant/generator itemsets)** và **tập mục sinh lời (productive itemsets)** có thể cung cấp thông tin chi tiết hơn.
 - Một tập mục là **không dư thừa** nếu không có tập con thực sự nào của nó có cùng support.
 - Một tập mục là **sinh lời** nếu tất cả các cách phân hoạch nó thành hai phần đều cho thấy mối tương quan tích cực giữa hai phần đó, nghĩa là support quan sát được của tập lớn hơn support kỳ vọng nếu giả định các phần độc lập. Các thuật toán như **Opus-Miner** và **IDPI+** (Interactive Discovery of Statistically Significant Itemsets) được phát triển để khai phá hiệu quả top-k các tập mục sinh lời, không dư thừa có ý nghĩa thống kê nhất.

Ngoài ra, nhiều độ đo tương quan khác như lift, cosine, coherence, và Kulczynski cũng được sử dụng rộng rãi, mỗi độ đo có những đặc tính và khả năng ứng dụng riêng, góp phần tạo nên một khung đánh giá thống nhất và toàn diện cho các mẫu tin khai phá được.

II. High-Utility Pattern Mining

Phần lớn, mục tiêu của các tổ chức dù lớn, vừa hay nhỏ đều nằm ở lợi nhuận. Vì thế nên, bên cạnh các thuật toán khai phá mẫu để tìm các mặt hàng phổ biến, hay các mặt hàng có thể kết hợp để tạo ra các túi hàng có thể thu hút được nhiều lượt mua hơn thông qua khai thác luật kết hợp (Association Rule Mining) thì việc khai phá những mẫu có lợi nhuận cao (High-Utility Pattern Mining - HUPM) sẽ là một bước tiến vượt bậc. Và những thuật toán sau đây, sẽ có đa dạng các cách tiếp cận về độ hữu ích (utility) để hỗ trợ tốt nhất cho những quyết định kinh doanh.

1. High Utility Itemset Mining

- a. Mục tiêu ?
 - Tìm những mẫu mang lại lợi nhuận cao, kể cả nó có phổ biến hay không (vì tuy có những mặt hàng lượt mua ít nhưng đem lại lợi nhuận cao).
- b. Ý tưởng chung:
 - Có một vấn đề đặt ra là: Lợi nhuận như thế nào là cao ? Nó sẽ phụ thuộc vào nhiều yếu tố như mặt hàng đang được kinh doanh, quy mô kinh doanh, độ uy tín của tổ chức. Từ đó, ý tưởng của việc khai phá những tập mục có lợi nhuận cao sẽ cho phép người dùng đặt ngưỡng lợi nhuận tối thiểu (minutil). Và sau khi có dữ liệu giao dịch (bao gồm cơ sở dữ liệu giao dịch : lưu những mặt hàng kèm theo cả số lượng trong từng giao dịch, và một bảng đơn vị lợi nhuận cho từng mặt hàng), thuật toán sẽ có ra những tập mục bằng hoặc lớn hơn minutil.
- c. Một số thuật toán cơ bản
 - Bởi vì lợi nhuận không phải dạng hàm tuyến tính, nghĩa là khi thêm một mặt hàng bất kỳ vào giỏ hàng đang có lợi nhuận có thể tăng, và cũng có thể giảm, việc tăng giảm lợi nhuận không phụ thuộc vào số lượng hàng được mua cho nên không thể áp dụng những thuật toán để khai thác những tập mục phổ biến (vì support là hàm tuyến tính).

- Một số thuật toán: Two-Phase (PAKDD 2005), IHUP (TKDE, 2010), UP-Growth (KDD 2011), HUI-Miner (CIKM 2012), FHM (ISMIS 2014).

✓ HUI-Miner (CIKM 2012): dùng phương pháp tìm kiếm sâu dạng cây (a depth-first search). Bằng cách tìm ngưỡng trên lợi nhuận của từng mặt hàng đơn lẻ, sau đó lại tính ngưỡng trên lợi nhuận khi kết hợp các mặt hàng đó lợi với nhau. Từ đó, ta sẽ biết được có nên kết hợp hay không, nếu việc kết hợp không tạo lợi nhuận cao hơn, ngưng và kết hợp cái khác, không cần tốn thời gian, dung lượng nhớ để làm điều không cần thiết. Trong quá trình xử lý dữ liệu, thuật toán này tìm kiếm những giao dịch nào có tập mục đó, tạo ra bảng sao nhỏ, sau tìm kiếm thì nối bảng có mặt hàng (tập mục) này với mặt hàng (tập mục khác), và điều này khiến cho nó khá đắt khi bước vào thực tế khi xử lý dữ liệu lớn.

✓ FHM (ISMIS 2014): sẽ khắc phục nhược điểm của HUI-Miner (CIKM 2012) bằng phương pháp Estimated-Utility Co-occurrence pruning (EUCS). EUCS có thể được cài đặt theo hai cách:

Ma trận tam giác (triangular matrix): chỉ lưu một nửa dữ liệu để tiết kiệm bộ nhớ (vì TWU của (a,b) = TWU của (b,a)).

Hashmap of hashmaps: dạng từ điển lồng nhau để truy xuất nhanh hơn.

✓ So sánh HUI-Miner (CIKM 2012), FHM (ISMIS 2014):

Thời gian: FHM nhanh gấp 6 lần HUI-Miner, nhưng với dữ liệu dày đặc thì hiệu suất thời gian 2 thuật toán này tương tự nhau.

Hiệu quả giảm không gian tìm kiếm, bộ nhớ : FHM tốt hơn

✓ The EFIM algorithm Items (Efficient High-Utility Itemset Mining): cũng áp dụng dùng phương pháp tìm kiếm sâu dạng cây (a depth-first search). Được hình thành từ 3 ý tưởng : HDP & Pattern-growth (Thu hẹp không gian tìm kiếm), HTM(Nén CSDL để xử lý nhanh hơn), Utility-Bin Array (UBA - Tra cứu và tính toán utility siêu nhanh). Trong khi khám phá không gian tìm kiếm, EFIM đặt ra thứ tự xử lý ban đầu sẽ giúp ta tránh việc khai thác một tập mục hai lần, điều này giúp giảm thời gian xử lý, giảm việc sử dụng dung lượng bộ nhớ, định nghĩa tập các mục mở rộng (Extension Set) - tức là những mục nào được phép "ghép thêm" vào một tập mục α đang xét để tạo thành một tập mục mới, lớn hơn.

d. Một số định nghĩa, phép tính cần nắm:

- Lợi nhuận của một giao dịch ($TU(T_i)$), lợi nhuận của một mặt hàng trong một giao dịch ($u(i, T_i) = p(i) \cdot q(i, T_i)$), lợi nhuận của một tập mục trong một giao dịch ($u\{a, c\}, T_i$), lợi nhuận của một tập mục trong một cơ sở dữ liệu ($u\{a, c\}$), ngưỡng trên lợi nhuận của một tập mục nào đó ($TWU(\{a, c\})$), Items, Itemset, Transaction database (D), Transaction ID (TID), External utility (unit profit), Internal utility (quantity = $q(\text{item}, T_i)$), một tập mục có lợi nhuận cao ($u\{a, c\} \geq \text{minutil}$).

Khái niệm	Định nghĩa	Ứng dụng	Tầm quan trọng
Utility (u)	Tính toán chính xác lợi nhuận của một mặt hàng hoặc một combo trong một hóa đơn.	Combo {Bánh mì, Sữa} trong hóa đơn A đem về 25k lợi nhuận.	QUAN TRỌNG NHẤT. Vì đây là con số cuối cùng mà mọi chiến lược hướng đến.
Remaining Utility (re)	Ước tính nhanh "trần" lợi nhuận mà một hóa đơn cụ thể còn có thể mang lại nếu khách hàng mua thêm các mặt hàng khác.	Ví dụ: Khách đã mua Áo, hóa đơn này còn có thể tăng thêm tối đa 500k nếu họ mua thêm Quần và Giày.	ÍT QUAN TRỌNG HƠN. Cần biết nó tồn tại để hiểu cách máy tính tối ưu hóa, nhưng ít khi dùng trực tiếp để ra quyết định kinh doanh.
Local Utility (lu)	QUYẾT ĐỊNH CÓ NÊN "CHƠI" MẶT HÀNG NÀY KHÔNG. Giúp loại bỏ ngay những mặt hàng/món đồ mà dù có	Ví dụ: Cà pha (z) có lu thấp => Không nên đưa vào menu chính, dù kết	RẤT QUAN TRỌNG. Tiết kiệm thời gian và tiền bạc. Giống như việc nghiên cứu thị trường trước

	kết hợp với bất cứ thứ gì khác cũng không đủ lợi nhuận.	hợp với bánh ngọt hay sữa cũng không sinh lời.	khi quyết định kinh doanh một mặt hàng mới. (Think Big)
Sub-tree Utility (su)	QUYẾT ĐỊNH COMBO NÀO ĐÁNG ĐỀ ĐẨY MẠNH. Tìm ra những combo cụ thể có lợi nhuận cao ngay lập tức và có tiềm năng mở rộng.	Combo {Pizza, Coke} (z) có su cao => Nên làm combo khuyến mãi, đưa lên banner quảng cáo.	RẤT QUAN TRỌNG. Giúp tìm ra "con gà đẻ trứng vàng" của cửa hàng. Đây là thứ em sẽ dùng để triển khai các chiến dịch marketing cụ thể.
Primary Items (p)	Danh sách "Ứng viên sáng giá". Những mặt hàng chắc chắn nên được gộp chung để tạo combo siêu lợi nhuận.	Khi khách mua Sữa, Bánh quy (P) là lựa chọn số 1 để gợi ý kèm theo.	QUAN TRỌNG. Đây là "list nhắm mục tiêu". Dùng để thiết kế các chương trình khuyến mãi, gợi ý sản phẩm hiệu quả nhất.
Secondary Items (s)	Danh sách "Hỗ trợ đắc lực". Những mặt hàng dùng để mở rộng các combo có sẵn, biến combo nhỏ thành combo lớn giá trị cao.	Từ combo {Sữa, Bánh quy}, có thể gợi ý thêm Ốc quế (S) để tạo thành Combo Bữa Sáng đắt giá hơn.	QUAN TRỌNG. Giúp tăng giá trị đơn hàng (up-selling). Biết được nên gợi ý thêm món gì sau khi khách đã chọn món chính.

2. *Minimal High-Utility Itemsets (the MinFHM algorithm)*

a. Là gì ?

- High-utility itemset mining mặc dù tạo ra được những tập mục mang lợi nhuận cao (vượt ngưỡng minutil) nhưng nó vẫn có một số nhược điểm sau: tạo ra số lượng quá lớn và một số thì dư – đây là vấn đề mà khai phá tập mục thường xuyên gặp phải. Để giải quyết vấn đề này, một lần nữa ta lại gặp khái niệm concise representation of HUIs (maximal, closed, generator)

The MinFHM algorithm (MinHUI) = High-utility itemset mining + minimal

CHUD = High-utility itemset mining + closed

GHUI-Miner = High-utility itemset mining + generator

Những thuật toán này không chỉ đảm bảo cho việc chỉ tìm ra những mẫu có lợi nhuận cao mà còn đảm bảo tính gọn (compact). Vì thế, tốc độ thuật toán cũng nhanh hơn.

3. *Mining Correlated High Utility Patterns (the FCHM algorithm)*

- Các thuật toán HUIM như HUI-Miner giúp phát hiện tập mục có lợi nhuận cao, nhưng đôi khi lại sinh ra những kết hợp “ảo” (spurious itemset). Chẳng hạn, từ A sang B có thể tạo ra lợi nhuận lớn, nhưng trong 100 giao dịch chỉ xuất hiện đúng 1 lần, cho thấy đây là sự kết hợp ngẫu nhiên chứ không phản ánh hành vi mua hàng thực tế. Những tập ảo như vậy tuy vượt ngưỡng lợi nhuận nhưng thiếu giá trị ứng dụng trong kinh doanh. Để khắc phục hạn chế này, thuật toán FCHM được

đề xuất, nhằm kết hợp cả hai yếu tố: lợi nhuận cao và mối tương quan thực sự giữa các sản phẩm, từ đó mang lại những mẫu khai thác sát với thực tiễn hơn.

- Một số tiếp cận để loại bỏ tập ảo là dùng kiểm định thống kê, thước đo affinity (so sánh mức độ xuất hiện cùng nhau so với riêng lẻ) và thước đo bond với công thức:

$$\text{bond}(X) = \text{conj_sup}(X) / \text{disj_sup}(X)$$

- Trong FCHM, mỗi itemset còn được lưu bằng conjunctive bitvector để biểu diễn nhanh giao dịch chứa nó, giúp tăng tốc tính toán và phát hiện tập vừa có utility cao vừa có tương quan thực sự.

- FCHM algorithm nhanh hơn lên đến 100 lần so với FHM.

4. TKQ:

TKQ = Mining top-k quantitative high utility itemsets + CHUQI-Miner: Mining correlated quantitative high utility itemsets.

- Bắt đầu từ bài toán khai thác **HUIs (High-Utility Itemsets)**, người ta mở rộng sang **HUQIs (High-Utility Quantitative Itemsets)** để giữ lại thông tin số lượng. Tuy nhiên, nếu phải xét từng số lượng chính xác (**Exact Q-itemsets**) thì chi phí tính toán rất cao. Vì vậy, có cách tiếp cận khác là dùng **Range Q-itemsets**, tức là gán cho mỗi mục một khoảng (lower,upper)(lower, upper)(lower,upper) thay vì duyệt từng giá trị riêng lẻ (ví dụ: (A,5),(B,2-4)(A,5), (B,2-4)(A,5),(B,2-4)). Các thuật toán như **FHUQI-Miner** và **CHUQI-Miner** dựa vào ý tưởng này, đồng thời tận dụng cấu trúc **prefix và join** để mở rộng dần các tập, nên vừa giảm được số trường hợp cần xét vừa vẫn tìm ra các tập có lợi nhuận cao.

- Tuy vậy, nhược điểm chung là vẫn phải đặt trước một ngưỡng **minutil**, điều này không dễ xác định và đôi khi bỏ sót thông tin hữu ích. Để khắc phục, có thêm hướng **TKQ (Top-K Quantitative Itemsets)**, cho phép tìm trực tiếp ra **K tập mục định lượng có utility cao nhất** mà không cần ngưỡng. Nhờ đó, không gian tìm kiếm được thu hẹp đáng kể, các nhánh ít tiềm năng bị loại bỏ sớm, giúp việc khai thác nhanh hơn và hiệu quả hơn.

5. Cross-level high utility itemset mining

$$\text{HUIs} \rightarrow \text{HUQIs} \rightarrow \text{TKQ} \rightarrow \text{MLHUI} \rightarrow \text{CLHUI} \rightarrow \text{TKC}$$

Một hạn chế lớn của việc khai thác **HUIs (High-Utility Itemsets)** là chỉ tập trung vào lợi nhuận, mà quên mất rằng hàng hóa trong thực tế thường được sắp xếp theo **cấu trúc phân cấp (taxonomy)**, ví dụ: *Food* $\rightarrow$ *Wheat* $\rightarrow$ *Bread* hoặc *Beverage* $\rightarrow$ *Coke/Water*. Nếu chỉ xét từng món lẻ, ta bỏ lỡ những thông tin ở mức khái quát cao hơn. Vì vậy, thuật toán **MLHUI-Miner** được đưa ra để khai thác các tập mục có lợi nhuận cao ở nhiều mức phân cấp khác nhau. Chẳng hạn, thay vì chỉ biết *Bread* có utility, ta có thể tính utility chung của cả nhóm *Wheat food*.

Tuy nhiên, MLHUI-Miner chỉ xử lý từng tầng riêng lẻ, chưa tìm ra được các kết hợp **giữa nhiều tầng**. Điều này dẫn đến sự ra đời của **CLHUI (Cross-Level High-Utility Itemsets)**, ví dụ như tập {Bread,Steak,Beverage}\{Bread, Steak, Beverage\} \{Bread,Steak,Beverage\}. Đây là những tập hữu ích vì phản ánh hành vi mua hàng thực tế, dù chúng không nằm trọn trong một cấp taxonomy nào.

Để sinh ra các tập như vậy, hai cơ chế mở rộng được dùng:

- **Join extension:** giống như “ghép thêm” các mục cùng cấp. Ví dụ: từ {Bread} có thể nối thêm {Steak} để thành {Bread,Steak}.
- **Tax extension:** là mở rộng xuống các cấp thấp hơn trong cây taxonomy. Ví dụ: từ {Food} có thể mở rộng ra {Bread} , {Pasta}.

Cả hai cách này được quản lý trong một cấu trúc gọi là **tax-utility-list**, nơi lưu lại thông tin về utility của các mục theo từng nhánh taxonomy. Nhờ đó, ta không cần duyệt mù quáng toàn bộ không gian tìm kiếm, mà có thể dùng các thước đo như **GWU (Generalized Weighted Utility)** và **REU (Remaining Expected Utility)** để sớm loại bỏ những nhánh không hứa hẹn.

Dẫu vậy, vẫn còn một khó khăn chung: phải đặt trước ngưỡng **minutil**. Nếu chọn cao quá thì bỏ sót nhiều mẫu quan trọng, còn chọn thấp thì tốn tài nguyên. Để giải quyết, cách tiếp cận **TKC (Top-K Cross-Level)** được phát triển. Thay vì phải “đoán” ngưỡng, TKC cho phép hệ thống tự động tìm ra **K tập cross-level có utility cao nhất**, vừa thực tiễn hơn, vừa tiết kiệm chi phí tính toán.

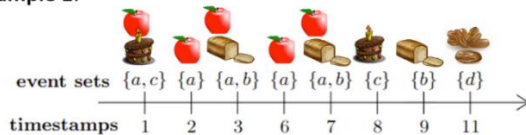
III. Episode Mining

a. Một số định nghĩa cơ bản:

Event sequence

An event sequence is an ordered list of event pairs $S = \langle (SE_{t_1}, t_1), (SE_{t_2}, t_2), \dots, (SE_{t_n}, t_n) \rangle$ where for any i , $SE_{t_i} \subseteq E$ is the set of events observed at time t_i .

Example 1:



$s = \langle (\{a, c\}, 1), \{a\}, 2, (\{a, b\}, 3), (\{a\}, 6), (\{a, b\}, 7), (\{c\}, 8), (\{b\}, 9), (\{d\}, 11) \rangle$

Episode

(the general case)

A (composite) episode $\alpha = \langle X_1, X_2, \dots, X_p \rangle$ is a list of event sets ordered by time, that is for any integers $1 \leq i < j \leq p$, X_i appeared before X_j .

Example: $\alpha = \langle \{a\}, \{a, b\}, \{c\} \rangle$



Apple was purchased. Then, apple and bread were bought at the same time, and then cake was purchased.

- Event types($E = \{a. b. c. d\}$), Event set ($\{a.b\}$), Event sequence, ($S = \langle (Set1, t1), (Set2, t2), \dots, (Setn, tn) \rangle$), A (composite) episode (gồm 2 loại: Episode tuần tự = Serial episode, Episode song song = Parallel episode), $occSet(\alpha)$ (all occurrences of an episode), head support with window (winlen)
- Support của episode: có nhiều phương pháp tiếp cận:
 - ✓ Window-based frequency
 - ✓ Head support (head frequency)
 - ✓ Total frequency
 - ✓ Non interleaved frequency
 - ✓ Minimal-occurences based frequency
 - ✓ ...

b. Một số thuật toán cơ bản :

- Frequent episode mining : input (an event sequence, two parameters: winlen, minsup), output (All frequent episodes: support $\geq$ minsup).
 - ✓ WINEPI (1995)
 - ✓ MINEPI (1995)
 - ✓ EMMA and MINEPI+(2008)

EMMA (2008) là một trong những thuật toán tiên phong khai thác frequent episodes trong chuỗi sự kiện phức tạp. Thuật toán dùng head support, tìm kiếm theo chiều sâu, và dựa trên cấu trúc dữ liệu dọc.

 - Bước đầu, dữ liệu được quét để tính support của các sự kiện đơn; những sự kiện có support $<$ minsup sẽ bị loại.
 - Với các sự kiện còn lại, EMMA xây dựng Location List – ghi lại vị trí (thời điểm) mà sự kiện xuất hiện. Support của một sự kiện đơn chính là số phần tử trong danh sách này.

- Từ các Location List, thuật toán tạo sự kiện phức tạp hơn (song song, nối tiếp) bằng cách giao các Location List và tiếp tục tính support. Quá trình mở rộng này lặp lại cho đến khi không còn tạo được tập mới.
- Để xử lý nhanh hơn, chuỗi dữ liệu được mã hoá bằng chỉ số sự kiện, đồng thời duy trì thêm Bound-List để giới hạn phạm vi mở rộng và cắt tỉa ứng viên không cần thiết.

Kết quả là tập các frequent episodes thỏa minsup được khai thác một cách hiệu quả.

✓ TKE(2019)

TKE = EMMA + TOP-K

Ý tưởng chính: TKE bắt đầu tìm kiếm với ngưỡng support = 1, sau đó tăng dần ngưỡng nội bộ cho đến khi thu được đúng K episode phổ biến nhất.

✓ AFEM, MaxFEM(2022)

MaxFEM(2022)

MaxFEM = EMMA + Maximal . Có một số phương pháp tối ưu nhưL

Opt 1: EFE (Efficient Filtering of Non-maximal episodes) loại non-maximal sớm.

Opt 2: SEC (Skip Extension Checking) bỏ qua mở rộng vô ích.

Opt 3: TP (Temporal Pruning) tỉa dựa vào ràng buộc thời gian.

AFEM

Sau khi áp dụng các tối ưu như EFE, SEC, và TP, MaxFEM đã cải thiện đáng kể hiệu năng trong việc tìm maximal episodes. Tuy nhiên, một vấn đề còn tồn tại là khi dữ liệu chuỗi rất lớn và phức tạp, việc tính toán chính xác mọi tập phổ biến vẫn tốn kém. Trong nhiều ứng dụng thực tế, đôi khi ta không cần tất cả kết quả chính xác tuyệt đối, mà chỉ cần những tập gần đúng nhưng đủ tin cậy để hỗ trợ phân tích. Chính nhu cầu đó đã dẫn đến AFEM (Approximate Frequent Episode Mining) – một hướng tiếp cận sử dụng xấp xỉ để khai thác nhanh hơn, chấp nhận hy sinh một phần nhỏ độ chính xác để đổi lấy tốc độ và khả năng xử lý dữ liệu quy mô lớn.

✓ POER , POER-ALL

Các nghiên cứu về quy tắc tập sự kiện (episode rules) đã phát triển qua nhiều giai đoạn, từ Utility-based episode rules (ASOC 2017) đến Precise-positioning episode rules (TKDE 2018) và Distant and essential episode rules (ESWA 2018). Các phương pháp này chủ yếu tập trung vào temporal ordering (thứ tự thời gian nghiêm ngặt), nhưng trong nhiều ứng dụng thực tế (như phân tích giao dịch bán lẻ), các quy tắc này có thể có ý nghĩa tương đương khi bỏ qua một số ràng buộc thứ tự. Điều này dẫn đến nhu cầu phát triển Partially-Ordered Episode Rules (POER) để khắc phục hạn chế của các phương pháp truyền thống.

POER (Partially-Ordered Episode Rules)

Định nghĩa: POER cho phép các sự kiện trong một tập có thứ tự một phần, nghĩa là chỉ một số cặp sự kiện có ràng buộc thứ tự, trong khi các cặp khác có thể tự do về thứ tự.

Ưu điểm:

Tính linh hoạt cao: Phù hợp với các ứng dụng where thứ tự sự kiện không hoàn toàn nghiêm ngặt (ví dụ: giao dịch bán lẻ, sự kiện mạng).

Giảm số lượng quy tắc dư thừa: Chỉ tập trung vào các quy tắc có ý nghĩa với thứ tự một phần.
 Hiệu quả trong dự báo: Có thể áp dụng cho các tác vụ dự báo trong nhiều lĩnh vực như quan sát thời tiết, xâm nhập mạng và thương mại điện tử.
 Nhược điểm:
 Độ phức tạp tính toán: Việc khai thác các quy tắc với thứ tự một phần đòi hỏi thuật toán tối ưu để giảm thiểu không gian và thời gian xử lý.

POER-ALL

Định nghĩa: POER-ALL là một biến thể của POER, khai thác tất cả các quy tắc tập sự kiện có thứ tự một phần, bao gồm cả các quy tắc có thể không đáng kể hoặc trùng lặp

IV. Sequential Pattern Mining

a. Mục đích:

- Khai thác và tìm ra tất cả các chuỗi con xuất hiện thường xuyên trong một chuỗi rời rạc (discrete sequence). Nói cách khác, SPM giúp nhận diện các mẫu tuần tự phổ biến trong dữ liệu, ví dụ như chuỗi hành vi khách hàng, nhật ký truy cập web, hoặc các sự kiện trong hệ thống. Một số định nghĩa, phép tính cơ bản

b. Một số định nghĩa, phép tính cơ bản cần nắm:

- A sequence (là một chuỗi biểu tượng có thứ tự : $S = \langle X_1, X_2, \dots, X_n \rangle$), Subsequence , Support of a sequence, Sequence database

c. Một số thuật toán cơ bản:

Sequential Pattern Mining (SPM):

- Input là sequence database và ngưỡng minsup; thuật toán tìm các chuỗi con xuất hiện đủ thường xuyên trong dữ liệu.
- Output là tập các sequential patterns phản ánh những chuỗi sự kiện lặp lại phổ biến.
- Nhưng để đảm bảo hiệu suất, thì thông thường sẽ không lấy hết lượng output đầu ra mà sẽ sử dụng một số kỹ thuật để lọc.

Một số thuật toán phổ biến:

- ✓ GSP
- ✓ SPAM
- ✓ SPADDE
- ✓ PrefixSpan

PrefixSpan (Prefix-Projected Sequential Pattern Mining) là một thuật toán phổ biến và hiệu quả cho bài toán khai thác các mẫu tuần tự. PrefixSpan hoạt động dựa trên nguyên lý chiếu cơ sở dữ liệu (projected database) kết hợp với tìm kiếm theo chiều sâu (depth-first search).

Cách thức hoạt động của thuật toán có thể tóm lược như sau: thay vì quét toàn bộ cơ sở dữ liệu (CSDL) gốc nhiều lần, PrefixSpan lần lượt xác định các tiền tố (prefix) phổ biến. Với mỗi tiền tố như vậy, nó tạo ra một CSDL chiếu – một CSDL con nhỏ hơn chỉ chứa các chuỗi có chứa tiền tố đó và các phần hậu tố đi kèm. Toàn bộ quá trình khai thác sau đó chỉ cần diễn ra trên các CSDL chiếu nhỏ này, giúp giảm đáng kể không gian tìm kiếm và chi phí tính toán. Nhờ cơ chế này, tốc độ xử lý của PrefixSpan gần như tuyến tính đối với kích thước của CSDL.

Mặc dù không phải là thuật toán hiệu quả nhất cho mọi trường hợp, ưu điểm lớn của PrefixSpan nằm ở sự đơn giản và dễ mở rộng. Tuy nhiên, hiệu suất của thuật toán chịu ảnh hưởng bởi nhiều yếu tố như: số lượng và độ dài của các chuỗi trong CSDL, số lượng mặt hàng (items) khác nhau, và đặc biệt là giá trị của ngưỡng hỗ trợ tối thiểu (min_sup). Một ngưỡng min_sup quá thấp có thể dẫn đến việc tạo ra số lượng lớn các mẫu và CSDL chiếu, làm tăng thời gian và bộ nhớ tiêu thụ.

✓ CM-SPAM and CM-SPADE

Nổi tiếp và nâng cao những thuật toán tìm những tập mục có lợi nhuận cao, nhưng quên xét yếu tố chi phí. Làm sao để có hiệu suất chi phí? Câu hỏi này được giải quyết theo 2 hướng tiếp cận phụ thuộc vào dữ liệu lợi nhuận là dạng nhị phân hay dạng số.

Bài toán 1: Khai thác mẫu hiệu quả chi phí với lợi ích nhị phân

Trong hướng tiếp cận này, lợi ích của một mẫu được xem là nhị phân (binary), ví dụ như việc một bệnh nhân "khỏi bệnh" hay "tử vong" sau một phác đồ điều trị, một chiến dịch marketing "thành công" hay "thất bại".

• Các độ đo quan trọng:

- ✓ Chi phí của một mẫu p ($\text{cost}(p)$): Tổng chi phí của các mặt hàng trong mẫu.
- ✓ Chi phí trung bình của một mẫu p ($\text{ac}(p)$): Được tính bằng $\text{cost}(p) / \text{sup}(p)$, trong đó $\text{sup}(p)$ là độ hỗ trợ (số lần xuất hiện) của mẫu. Chỉ số này phản ánh chi phí bỏ ra trên mỗi lần mẫu đó xảy ra. Một mẫu có $\text{ac}(p)$ thấp được ưu tiên hơn.
- ✓ Hệ số tương quan ($\text{cor}(p)$): Đo lường mối quan hệ thống kê giữa sự xuất hiện của mẫu p và một kết quả tích cực (lợi ích nhị phân). Một mẫu có hệ số tương quan cao và chi phí trung bình thấp là rất có giá trị.

• Giải pháp:

Để khai thác hiệu quả các mẫu này từ cơ sở dữ liệu chuỗi, hai thuật toán tiêu biểu là CM-SPADE và CM-SPAM được đề xuất. Các thuật toán này kế thừa và mở rộng từ các thuật toán khai thác mẫu tuần tự phổ biến là SPADE và SPAM, tích hợp thêm các độ đo về chi phí và hệ số tương quan để lọc và đánh giá các mẫu, từ đó tìm ra những mẫu vừa có ý nghĩa thống kê vừa tiết kiệm chi phí.

Problem 1: Finding all cost-effective patterns

Sid	<(Event:cost)>	Utility
S ₁	<(a:4)(b:2)(e:4)(c:4)(d:5)>	Positive
S ₂	<(b:3)(c:2)(f:1)(d:1)(e:2)>	Negative
S ₃	<(a:2)(f:2)(e:1)(c:3)(d:5)>	Positive
S ₄	<(a:2)(b:2)(c:1)(f:2)>	Negative

A pattern p is **cost-effective** if:

$$\text{sup}(p) \geq \text{minsup}$$

$$\text{ac}(p) \leq \text{maxcost}$$

And we measure the **correlation** of a pattern p with the desirable outcome:

$$\text{cor}(p) = \frac{\text{ac}(D_p^+) - \text{ac}(D_p^-)}{\text{Std}} \sqrt{\frac{|D_p^+||D_p^-|}{|D_p^+ \cup D_p^-|}} \in [-1, 1]$$

a positive correlation is desirable

Pattern	support	average cost	correlation
<ac>	3	5.3	0.80

More details...

The **correlation** of a pattern p :

$$\text{cor}(p) = \frac{\text{ac}(D_p^+) - \text{ac}(D_p^-)}{\text{Std}} \sqrt{\frac{|D_p^+||D_p^-|}{|D_p^+ \cup D_p^-|}}$$

where, $\text{ac}(D_p^+)$, $\text{ac}(D_p^-)$ denotes pattern p 's average cost in positive and negative sequences, respectively

- $\text{ac}(D_p^+) - \text{ac}(D_p^-)$, indicates the difference in terms of average cost of positive and negative sequences.
- Std , standard deviation of the cost to avoid absolute values.
- $\sqrt{\frac{|D_p^+||D_p^-|}{|D_p^+ \cup D_p^-|}}$, measures distribution difference to indicate patterns effect on the outcome.
- Correlation values are in the $[-1, 1]$ interval.
- The greater positive(negative) the cor measure is, the more pattern is correlated with a positive (negative) utility.

Bài toán 2: Khai thác mẫu hiệu quả chi phí với lợi ích dạng số

Ở bài toán phức tạp hơn này, lợi ích (utility) của mỗi mẫu hàng là một giá trị số cụ thể (ví dụ: lợi nhuận tính bằng tiền). Mục tiêu là tìm ra những mẫu có sự cân đối tốt nhất giữa lợi ích và chi phí.

- Độ đo quan trọng:
 - ✓ Lợi ích của một mẫu p ($u(p)$): Tổng lợi ích mà mẫu đó mang lại.
 - ✓ Chỉ số đánh đổi (trade-off) của một mẫu p ($tf(p)$): Được định nghĩa là tỷ lệ giữa chi phí trung bình và lợi ích, $tf(p) = ac(p) / u(p)$. Chỉ số này càng thấp càng tốt, vì nó thể hiện ta chỉ phải bỏ ra một chi phí trung bình nhỏ để đạt được một đơn vị lợi nhuận.

- Giải pháp:

Một số thuật toán được phát triển để giải quyết bài toán này, tiêu biểu là:

- ✓ AMSC (Mining Affordable and Manageable Cost-cost-effective patterns): Tập trung vào việc khai thác các mẫu có chi phí trung bình nằm trong một ngưỡng cho phép do người dùng định nghĩa, đồng thời có lợi ích cao.
- ✓ CorCEPB (Correlated Cost-Effective Pattern mining with Binary utility): Mặc dù tên có chứa "Binary", khái niệm này có thể được mở rộng để đánh giá mối tương quan giữa chi phí và lợi ích số.
- ✓ CEPN (Cost-Effective Pattern mining with Numeric utility): Thuật toán trực tiếp giải quyết bài toán bằng cách sử dụng chỉ số đánh đổi $tf(p)$ làm thước đo chính để đánh giá và lựa chọn các mẫu hiệu quả nhất.

V. Sequential Rule Mining

Từ CMRules, CMDeo đến RuleGrowth: Sự Phát Triển Của Các Thuật Toán Khai Thác Luật Tuần Tự

Hành trình phát triển các thuật toán khai thác luật tuần tự bắt đầu với **CMRules** - phương pháp tiếp cận đầu tiên. CMRules dựa trên một quan sát quan trọng: luật tuần tự là một dạng đặc biệt của luật kết hợp. Thuật toán này hoạt động theo hai pha: đầu tiên, sử dụng một thuật toán khai thác luật kết hợp (như Apriori) trên một cơ sở dữ liệu giao dịch thu được bằng cách bỏ qua thông tin thứ tự thời gian; sau đó, nó lọc các luật kết hợp tìm được bằng cách quét lại cơ sở dữ liệu tuần tự gốc để kiểm tra tính tuần tự và loại bỏ những luật không đạt ngưỡng hỗ trợ và tin cậy tuần tự.

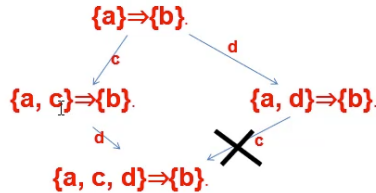
Mặc dù đơn giản, CMRules có một nhược điểm lớn: hiệu suất. Việc phải sinh ra toàn bộ tập luật kết hợp trước khi lọc là rất tốn kém, vì nhiều luật trong số đó cuối cùng sẽ bị loại bỏ.

CMDeo (Cross-Mining with Data Execution Order) ra đời như một bước cải tiến, khắc phục hạn chế này. Thay vì dựa vào luật kết hợp, CMDeo trực tiếp khai thác luật tuần tự ngay từ đầu. Nó sử dụng chiến lược "khai thác chéo" (cross-mining), tận dụng thông tin về thứ tự dữ liệu để khám phá các luật một cách hiệu quả hơn, tránh việc phải tạo ra một lượng lớn ứng viên không cần thiết.

Kế thừa và phát triển những ưu điểm của CMDeo, **RuleGrowth** đã trở thành một trong những thuật toán hiệu quả nhất cho bài toán này. RuleGrowth hoạt động dựa trên nguyên lý **mở rộng** (expansion). Cụ thể, thuật toán này phát triển các luật tuần tự bằng cách:

How to avoid generating the same rules twice?

Problem 1: The same rule can be generated by adding items in different orders.

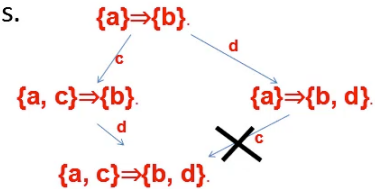


Solution: Add only an item to the left/right part of a rule if the item is larger than all items already in the left/right part.

22

How to avoid generating the same rules twice?

Problem 2: the same rule can be generated by adding items in different orders of left/right expansions.



Solution: Do not allow performing a left expansion after a right expansion. But allow performing a right expansion after a left expansion.

23

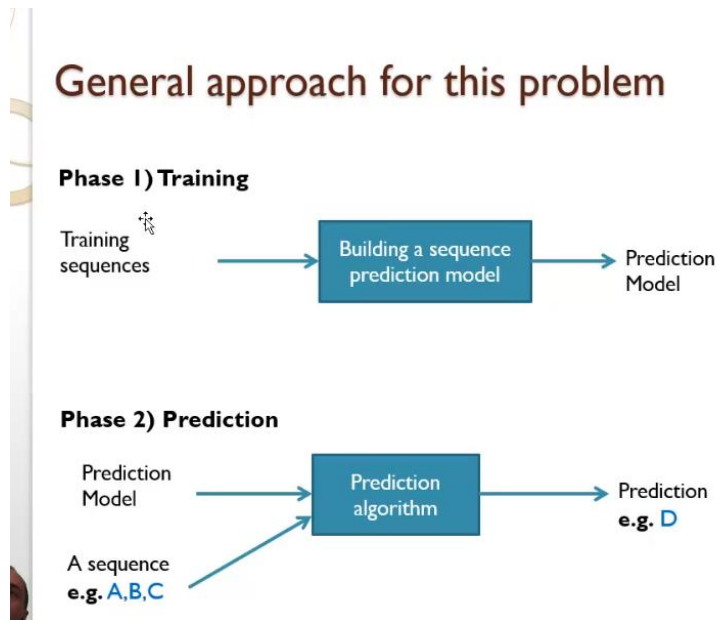
- **Mở rộng bên phải (Right Expansion):** Thêm một itemset vào phần hậu tố (Y) của luật $X \rightarrow Y$.
- **Mở rộng bên trái (Left Expansion):** Thêm một itemset vào phần tiền đề (X) của luật $X \rightarrow Y$.

Để tránh việc cùng một luật được sinh ra nhiều lần, RuleGrowth áp dụng các nguyên tắc quan trọng:

1. **Thứ tự lexical:** Chỉ thêm một item vào phần left hoặc right của luật nếu item đó có giá trị lớn hơn (theo thứ tự lexical) tất cả các items đã có trong phần đó.
2. **Thứ tự mở rộng:** Chỉ cho phép mở rộng bên phải **sau** khi đã thực hiện mở rộng bên trái, chứ không cho phép thực hiện theo chiều ngược lại.

Nhờ chiến lược mở rộng thông minh và tập trung hoàn toàn vào cấu trúc tuần tự, RuleGrowth loại bỏ hoàn toàn giai đoạn sinh luật kết hợp tốn kém của CMRules, mang lại hiệu suất vượt trội và trở thành thuật toán tiêu chuẩn để giải quyết bài toán khai thác luật tuần tự.

VI. Sequence Prediction (source code version only)



a. Mục tiêu:

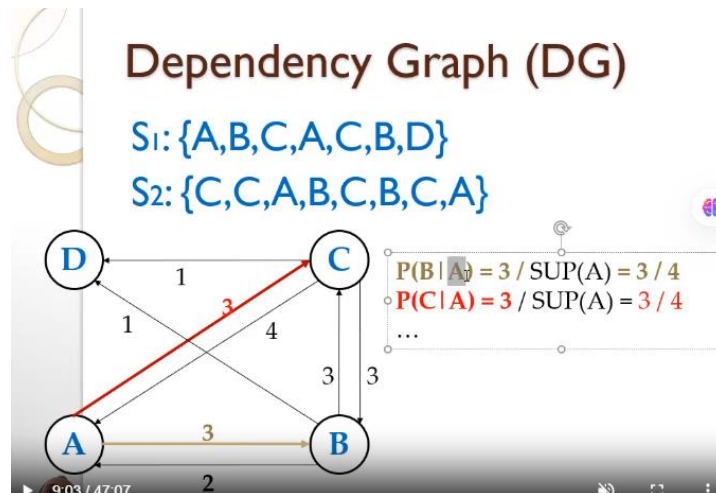
Dự đoán ký hiệu (symbol) tiếp theo trong một chuỗi dựa trên các chuỗi dữ liệu huấn luyện (training sequences).

b. Ứng dụng: Ứng dụng rộng rãi trong thực tế như:

- Dự đoán trang web tiếp theo để tải trước (prefetching)
- Phân tích hành vi người dùng trên website
- Hỗ trợ gõ chữ (keyboard prediction)
- Hệ thống gợi ý sản phẩm (product recommendation)
- Dự báo thị trường chứng khoán (stock market prediction)

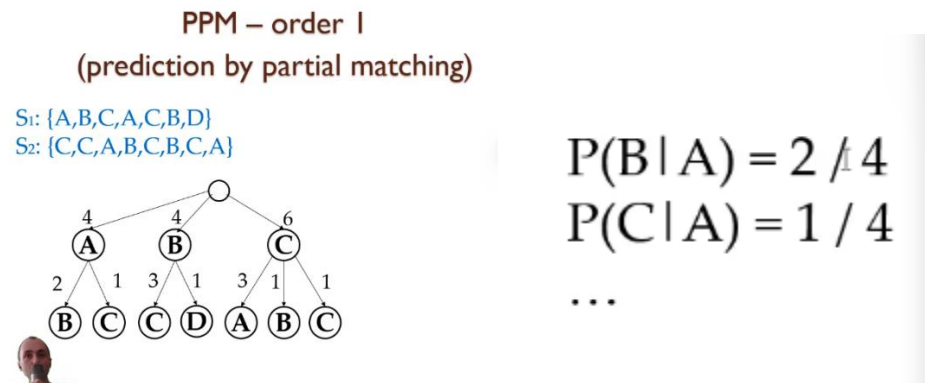
c. Các Phương Pháp Truyền Thống & Hạn Chế

- **Khai thác mẫu tuần tự (Sequential Pattern Mining - e.g., PrefixSpan):**
 - **Cách làm:** Khai thác các mẫu phổ biến (ví dụ: a, e với độ hỗ trợ 66%) từ dữ liệu để dùng cho dự đoán.
 - **Hạn chế:** Tốn thời gian, bỏ qua các trường hợp hiếm, và rất tốn kém để cập nhật mô hình khi có dữ liệu mới.
- **Đồ thị phụ thuộc (Dependency Graph - DG):** Tính xác suất xảy ra của ký hiệu này sau một ký hiệu khác (ví dụ: $P(B|A)$).

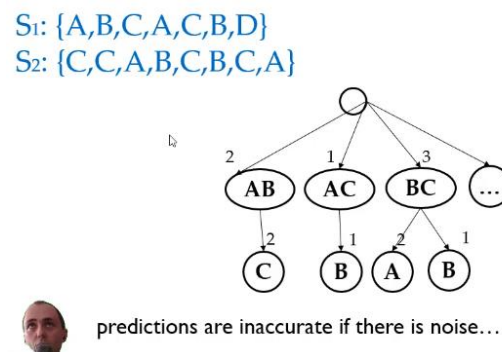


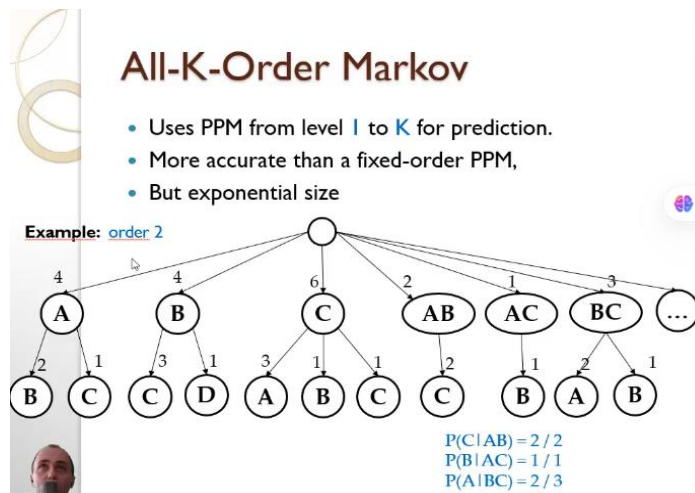
- Mô hình Markov (PPM - Prediction by Partial Matching):**

- Dự đoán dựa trên k ký hiệu đứng ngay trước (bậc k).
- **Hạn chế:** Độ chính xác giảm nếu có nhiễu (noise). Mô hình bậc cao (All-K-Order Markov) cho độ chính xác tốt hơn nhưng kích thước mô hình tăng theo cấp số nhân, dẫn đến chi phí tính toán rất lớn.



PPM – order 2





d. Tổng kết hạn chế của các mô hình cũ:

- Giả định không thực tế (ví dụ: sự kiện chỉ phụ thuộc vào sự kiện ngay trước nó).
- Kích thước mô hình lớn hoặc độ chính xác thấp.
- Kém khả năng chống nhiễu.
- Khó cập nhật.
- Là mô hình **không đầy đủ (lossy)**, tức là chúng bỏ mất một phần thông tin từ dữ liệu huấn luyện.

e. Giải Pháp: Mô Hình CPT (Compact Prediction Tree)

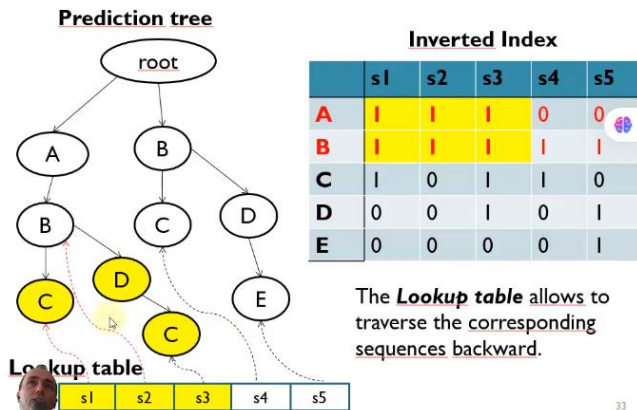
Để khắc phục những hạn chế trên, mô hình **Cây Dự đoán Nén (CPT)** được đề xuất với mục tiêu:

- **Tăng độ chính xác** của dự đoán.
- Duy trì một **kích thước mô hình hợp lý**.
- **Chống nhiễu tốt**.
- **Giả thuyết then chốt:** Xây dựng một mô hình **đầy đủ (lossless)** - lưu trữ toàn bộ thông tin từ dữ liệu huấn luyện - và sử dụng tất cả thông tin liên quan cho mỗi lần dự đoán sẽ giúp tăng độ chính xác.

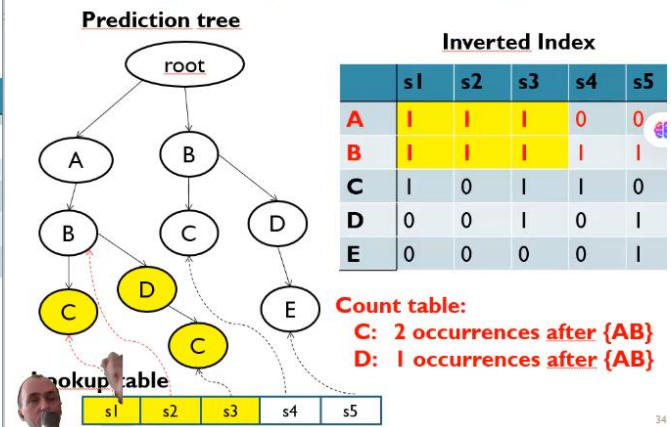
Các thách thức CPT giải quyết:

- Định nghĩa một **cấu trúc lưu trữ hiệu quả** về mặt bộ nhớ.
- Cấu trúc phải **dễ dàng cập nhật** khi thêm chuỗi dữ liệu mới.
- Đề xuất một **thuật toán dự đoán** vừa chính xác vừa hiệu quả về thời gian.
- **Cấu trúc của CPT:**

Predicting the symbol following <A,B>



Predicting the symbol following <A,B>



CPT là một cấu trúc cây để lưu trữ các chuỗi huấn luyện, kết hợp với một **cơ chế lập chỉ mục (inverted index)** và một **bảng tra cứu (lookup table)**. Cơ chế này cho phép truy ngược nhanh chóng các chuỗi chứa một ký hiệu cụ thể, giúp thuật toán dự đoán có đầy đủ thông tin để đưa ra kết quả chính xác nhất. Mỗi chuỗi dữ liệu mới được chèn vào cây một cách tuần tự, giúp mô hình dễ dàng được cập nhật.

f. CPT+ (Compact Prediction Tree Plus) - Tối ưu CPT

Mục tiêu: Giảm Độ phức tạp (Time/Space Complexity) của CPT.

3 Cải tiến chính:

Nén chuỗi con phổ biến

Nén nhánh đơn

Khử nhiễu cải tiến

II. Periodic pattern mining

1. Luật chu kỳ (Period rules)

Luật tuần tự (Sequential Rules):

Luật tuần tự tập trung vào việc phát hiện các mối quan hệ theo thứ tự thời gian, tức là sự kiện nào thường xảy ra trước và sự kiện nào thường xảy ra sau. Chẳng hạn, trong kinh doanh bán lẻ, người ta thường quan sát thấy rằng khi khách hàng mua điện thoại thông minh, họ thường sẽ mua ốp lưng hoặc tai nghe trong vòng vài ngày sau đó. Đây là một ví dụ gần gũi cho thấy tính chất nguyên nhân – kết quả theo trình tự trong dữ liệu.

Luật chu kỳ (Periodic Rules):

Ngược lại, luật chu kỳ nhấn mạnh đến việc phát hiện những mẫu lặp lại đều đặn theo thời gian, không cần có quan hệ nhân – quả trực tiếp, mà quan trọng là sự tuần hoàn. Ví dụ thực tiễn trong kinh doanh là việc doanh số bia và đồ nướng thường tăng cao vào cuối tuần, hoặc sản phẩm kem chống nắng luôn được tiêu thụ nhiều hơn vào mùa hè hằng năm. Đây là các quy luật có tính chất định kỳ, giúp doanh nghiệp dự báo và chuẩn bị nguồn lực cho các giai đoạn cao điểm.

2. Các loại mẫu phân loại theo chiều thời gian (time dimension)

a. Các loại mẫu

- Periodic Pattern (Mẫu chu kỳ):

Đây là những mẫu sự kiện lặp lại theo chu kỳ ổn định trong dữ liệu thời gian. Ví dụ, doanh số nước giải khát thường tăng cao vào mùa hè hằng năm, hoặc lượng khách tại các siêu thị luôn đạt đỉnh vào cuối tuần. Nhận diện các mẫu chu kỳ giúp doanh nghiệp tối ưu tồn kho và kế hoạch marketing theo mùa vụ.

- Peak Pattern (Mẫu đỉnh):
- Mẫu đỉnh nhấn mạnh vào những thời điểm có biến động mạnh, tăng vọt hoặc giảm sâu so với bình thường. Ví dụ, lượng truy cập website thương mại điện tử tăng đột biến trong dịp Black Friday hoặc 11/11. Việc nhận diện các mẫu đỉnh giúp doanh nghiệp chuẩn bị hệ thống và nguồn lực để tận dụng cơ hội hoặc ứng phó rủi ro.

Trending Pattern (Mẫu xu hướng):

- Mẫu xu hướng mô tả sự thay đổi tăng dần hoặc giảm dần theo thời gian, không nhất thiết có tính chu kỳ. Ví dụ, nhu cầu sử dụng ví điện tử có xu hướng tăng liên tục trong vài năm gần đây, hay doanh số đĩa CD giảm dần theo thời gian. Nhận diện xu hướng giúp doanh nghiệp điều chỉnh chiến lược dài hạn và nắm bắt thị trường mới.
3. **Khai thác tập mục có chu kỳ (Periodic Itemset Mining)**
- Mục tiêu: tìm các nhóm mặt hàng xuất hiện có chu kỳ trong một chuỗi các giao dịch.
 - Các định nghĩa cơ bản: periodicity of an itemset, maximum periodicity, Periodic Itemset

Definition: Periodicity

A transaction database

Transaction	Items
T ₁	{a, c}
T ₂	{e}
T ₃	{a, b, c, d, e}
T ₄	{b, c, d, e}
T ₅	{a, c, d}
T ₆	{a, c, e}
T ₇	{b, c, e}

Periods of an itemset:

the number of transactions between each occurrence of the itemset

Example:

The periods of the itemset {a,c} are
 $ps(\{a,c\}) = \{1, 2, 2, 1, 1\}$

Definition: Maximum periodicity

A transaction database

Transaction	Items
T ₁	{a, c}
T ₂	{e}
T ₃	{a, b, c, d, e}
T ₄	{b, c, d, e}
T ₅	{a, c, d}
T ₆	{a, c, e}
T ₇	{b, c, e}

Maximum periodicity:

The maximum periodicity of an itemset **X** is the maximum of its periods

$$\maxPer(X) = \max(ps(X))$$

Example:

The periods of the itemset {a,c} are
 $ps(\{a,c\}) = \{1, 2, 2, 1, 1\}$

$$\maxPer(X) = \max(\{1, 2, 2, 1, 1\}) = 2$$

Definition: Periodic Itemset

A transaction database

Transaction	Items
T ₁	{a, c}
T ₂	{e}
T ₃	{a, b, c, d, e}
T ₄	{b, c, d, e}
T ₅	{a, c, d}
T ₆	{a, c, e}
T ₇	{b, c, e}

Periodic itemset

An itemset **X** is periodic if
 $\maxPer(X) \leq \maxPer$

Example:

The periods of the itemset
 $\maxPer(\{a,c\}) = 2$

If $\maxPer = 1$, then {a,c} is **NOT** periodic

- Một số thuật toán cơ bản để khai thác periodic itemsets:
 Cách truyền thống: Dựa trên thuộc tính chính là “Nếu tập mục X không có chu kỳ thì bất cứ tập mở rộng Y của X thì cũng không có chu kỳ”.
 ✓ PF-Growth: dựa trên FPGrowth
 ✓ MTKPP: dựa trên Eclat

Nhưng định nghĩa mẫu có chu kỳ quá khắc khe, điều này sẽ dẫn đến trường hợp “Nesu một tập mục luôn có chu kỳ nhưng chỉ cần duy nhất một lần vượt qua maxPer thì nó sẽ bị loại bỏ”.

- ✓ PFP algorithm : để khắc phục vấn đề đó, Fournier-Viger đã tạo ra tham số đầu vào là một khoảng ngưỡng gồm có minAvg, maxAvg, minPer, maxPer. Hoạt động dựa trên ECLAT.

PFP nhanh hơn ECLAT bởi vì nó lọc nhiều mẫu không có chu kỳ (thời gian chạy ít hơn ECLAT gấp 4 hay 5 lần).

- ✓ SPP-Growth: khai phá những mẫu thường có chu kỳ ổn định trong dữ liệu giao dịch.

Thuật toán SPP-Growth được thiết kế để tìm các mẫu itemset vừa thường xuyên ($\text{support} \geq \text{minSup}$), vừa có tính ổn định chu kỳ (stability) cao. Tính ổn định được đánh giá qua hai khái niệm chính:

- Lability $la(X)$: tổng tích lũy của độ lệch (mỗi khoảng giữa hai lần xuất hiện) so với ngưỡng maxPer, nếu khoảng đó vượt maxPer thì lability tăng, nếu nhỏ hơn thì có thể giảm xuống (tối thiểu là 0). (Ví dụ, với $\text{per} = \{2,3,2,2,1\}$ và $\text{maxPer} = 2$ thì $la(X)$ từng bước: 0,1,1,1,0)
- Stability $\text{maxla}(X)$: là giá trị lớn nhất trong tất cả các $la(X,i)$, tức là mức lệch cực đại mà mẫu phải chịu. Một itemset được xem là stable periodic-frequent pattern (SPP) nếu thỏa mãn $\text{sup}(X) \geq \text{minSup}$ và $\text{maxla}(X) \leq \text{maxLa}$ (với maxLa là ngưỡng ổn định do người dùng đặt.)

Để giảm không gian tìm kiếm, SPP-Growth dùng hai tính chất:

- $\text{sup}(X)$ là anti-monotone (nếu $\text{sup}(X)$ nhỏ thì $\text{sup}(\text{superset})$ càng nhỏ)
- $\text{maxla}(X)$ là monotone (nếu mẫu mở rộng thêm item thì $\text{maxla}(\text{superset}) \geq \text{maxla}(X)$)
- Philippe Fournier-Viger

Thuật toán xây dựng trước cây SPP-tree tương tự FP-tree, nhưng lưu thêm thông tin TID để tính lability và maxla. Khi duyệt cây, nếu một mẫu X có $\text{maxla}(X)$ vượt ngưỡng maxLa thì loại bỏ cả nhánh X và các superset của nó khỏi tìm kiếm.

Philippe Fournier-Viger

So sánh với các thuật toán periodic-frequent truyền thống:

- Truyền thống loại bỏ mẫu nếu bất cứ một khoảng chu kỳ nào vượt maxPer, bất kể những lần khác như thế nào.
- Partial PFP được nói rộng hơn nhưng vẫn không xét bao nhiêu hoặc lệch bao nhiêu so với maxPer, nên mẫu có khoảng rất lớn vẫn được chấp nhận nếu chỉ có vài lần vượt.
- SPP-Growth giúp giải quyết điều này bằng cách đo lường lệch độ lớn qua lability, nên tìm được những mẫu chu kỳ ổn định hơn, có thể bỏ qua những lần “trật chu kỳ” nhỏ mà vẫn giữ mẫu nếu nhìn chung ổn định.
- ✓ Periodic high utility itemset mining and the PHM algorithm:
 - Một số nghiên cứu để đo periodicity: PFP-Tree, MKTPP, ITL-Tree, MaxCPF
 - PHM: hoạt động cũng tương tự như SPP-Growth nhưng kết hợp thêm điều kiện lợi nhuận cao.

VII. Time Series Mining

1. Định nghĩa:

Chuỗi thời gian là tập hợp các điểm dữ liệu được ghi lại theo thứ tự thời gian, xuất hiện trong mọi lĩnh vực từ tài chính, khí tượng đến y học. Dữ liệu thô thường chứa nhiều nhiễu, xu hướng (trend) và tính thời vụ (seasonality), khiến việc phân tích trực tiếp trở nên khó khăn. Bộ công cụ SPMF cung cấp một hệ thống phương pháp toàn diện để biến dữ liệu thô thành thông tin có giá trị.

2. Quy trình xử lý được đề xuất theo một logic tuần tự hợp lý:

Trực quan hóa & Làm mịn: Bước đầu tiên phải là trực quan hóa để cảm nhận hình dạng, nhiễu, xu hướng và điểm bất thường của dữ liệu. Sau đó, làm mịn được áp dụng để giảm nhiễu ngẫu nhiên, giúp ta nhìn thấy các tín hiệu cốt lõi (như xu hướng) một cách rõ ràng hơn mà chưa làm thay đổi cấu trúc cơ bản của dữ liệu.

Chuẩn hóa Dữ liệu: Sau khi có dữ liệu "sạch" hơn, bước này đảm bảo các chuỗi thời gian khác nhau được đặt trên cùng một thang đo. Điều này là bắt buộc để so sánh công bằng hoặc chuẩn bị dữ liệu cho các thuật toán học máy nhạy cảm với khoảng cách và độ lớn (như K-Means, Neural Network).

Biến đổi & Rút trích Đặc trưng: Ở bước này, chúng ta chủ động biến đổi dữ liệu để giải quyết các vấn đề như xu hướng hoặc rút gọn kích thước dữ liệu trong khi vẫn giữ lại những đặc trưng quan trọng nhất. Các đặc trưng được trích xuất sẽ là đầu vào cho các mô hình khai phá và dự báo.

Phân đoạn & Biểu diễn Ký hiệu: Đây là bước cao cấp, nơi dữ liệu số được chuyển đổi sang miền ký hiệu, cho phép áp dụng sức mạnh của các thuật toán Khai phá mẫu trình tự và Luật kết hợp (vốn được thiết kế cho dữ liệu dạng chữ) vào lĩnh vực chuỗi thời gian, mở ra khả năng phát hiện các mẫu hình phức tạp.

Quy trình này đi từ tiền xử lý cơ bản đến các biến đổi nâng cao, mỗi bước tạo nền tảng cho bước tiếp theo, nhằm mục đích cuối cùng là tạo ra dữ liệu đầu vào tối ưu cho các thuật toán khai phá tri thức.

2.1 Trực quan hóa và Làm mịn (Visualization & Smoothing)

Công việc đầu tiên và quan trọng nhất là trực quan hóa (Ví dụ 249) để nắm bắt hình dạng, xu hướng và điểm bất thường của chuỗi dữ liệu. Sau đó, các kỹ thuật làm mịn được áp dụng để giảm nhiễu, giúp ta nhìn thấy xu hướng cốt lõi dễ dàng hơn.

Trực quan hóa: Làm thế nào để nhận biết vấn đề?

Việc đầu tiên là vẽ dữ liệu thô ra một biểu đồ đường (line plot). Từ hình dạng của đường này, ta có thể "đọc" được những đặc tính cơ bản:

- **Nhiều (Noise):** Đường dữ liệu có nhiều gai nhọn, lên xuống hỗn loạn, không theo một quy luật rõ ràng nào trong ngắn hạn. Điều này cho thấy dữ liệu bị ảnh hưởng bởi các yếu tố ngẫu nhiên.
- **Xu hướng (Trend):** Đường dữ liệu có hướng đi lên hoặc đi down một cách rõ ràng trong một khoảng thời gian dài. Ví dụ, doanh số tăng trưởng đều qua các năm.
- **Tính thời vụ (Seasonality):** Xuất hiện các chu kỳ lặp đi lặp lại đều đặn. Ví dụ, cứ mỗi 7 ngày, hình dáng đường dữ liệu lại giống nhau (chu kỳ tuần), hoặc mỗi 12 tháng lại có một đỉnh (chu kỳ năm).
- **Điểm bất thường (Outliers/Anomalies):** Là những điểm dữ liệu nằm rất xa so với phần còn lại, không tuân theo bất kỳ hình mẫu nào. Ví dụ, một ngày doanh số đột ngột tăng vọt gấp 10 lần rồi lại trở về bình thường.

Vấn đề của dữ liệu thô & Khi nào cần làm mịn?

- Dữ liệu thô luôn chứa nhiễu. Làm mịn được sử dụng khi mục tiêu của chúng ta là nhìn thấy "tín hiệu" (signal) - tức là xu hướng và tính thời vụ cốt lõi - bên dưới "nhiều" (noise).
- Nếu gặp vấn đề về NHIỀU: -> Dùng làm mịn (Moving Average, Median, Exponential Smoothing) để loại bỏ bớt các biến động ngẫu nhiên, giúp đường xu hướng trở nên rõ ràng và mượt mà hơn.(bước 1)

- Nếu gặp vấn đề về XU HƯỚNG hoặc THỜI VỤ: -> Làm mịn KHÔNG giải quyết được triệt để. Khi đó, ta cần các phương pháp biến đổi mạnh tay hơn như Lấy sai phân (Differencing) hoặc Áp dụng các mô hình thống kê (như Decomposition) để tách riêng các thành phần này ra. (bước 2)

Xử lý nhiễu:

Phương pháp	Cách làm dành cho phương pháp tương ứng	Ưu, nhược điểm
1. Moving Average (Trung bình trượt)	Cách 1: Prior Moving Average (Trung bình trượt quá khứ - Ví dụ 250): Tính trung bình của một số điểm trong quá khứ. Ví dụ: Để làm mịn doanh số ngày 10/5, ta lấy trung bình doanh số từ ngày 7/5 đến 9/5 (cửa sổ 3 ngày).	Đặc điểm/Ưu điểm: Đơn giản, dễ hiểu. Nhược điểm: Có độ trễ (lag) vì chỉ dùng dữ liệu quá khứ, khiến đường làm mịn luôn "đi sau" xu hướng thực tế. Không dùng để dự báo điểm ngay tiếp theo được.
	Cách 2: Central Moving Average (Trung bình trượt trung tâm - Ví dụ 252): Tính trung bình các điểm xung quanh điểm hiện tại (cả quá khứ và tương lai). Ví dụ: Làm mịn nhiệt độ ngày 10/5 bằng cách lấy trung bình từ ngày 9/5 đến 11/5.	Đặc điểm/Ưu điểm: Kết quả làm mịn mượt và sát với xu hướng thực hơn vì nó nằm ở trung tâm của cửa sổ dữ liệu. Nhược điểm: Không thể dùng để dự báo vì cần dữ liệu trong tương lai. Chỉ phù hợp để phân tích xu hướng lịch sử.
	Cách 3: Cumulative Moving Average (Trung bình tích lũy - Ví dụ 251): Tính trung bình tích lũy của tất cả các điểm dữ liệu từ đầu đến điểm hiện tại. Ví dụ: Điểm GPA của một sinh viên chính là trung bình tích lũy của tất cả các môn học từ năm nhất.	Đặc điểm: Phản ánh toàn bộ lịch sử, nhưng càng về sau càng ít nhạy cảm với sự thay đổi mới vì mỗi điểm dữ liệu mới chỉ góp một phần rất nhỏ vào giá trị trung bình đã tích lũy lớn.

2. Median Smoothing (Làm mịn trung vị - Ví dụ 255): Thay vì dùng trung bình, phương pháp này dùng trung vị (giá trị nằm giữa khi sắp xếp dãy số) của một cửa sổ dữ liệu.	Tại sao giảm nhiễu tốt hơn? Trung bình rất nhạy cảm với các giá trị ngoại lai (outliers). Ví dụ, cửa sổ 5 điểm có giá trị [10, 11, 12, 13, **100**]. Trung bình là $(10+11+12+13+100)/5 = 29.2$ (sai lệch lớn so với thực tế). Trung vị là 12 - giá trị giữa sau khi sắp xếp - hoàn toàn không bị ảnh hưởng bởi giá trị 100.	Do đó, Median Smoothing ưu việt hơn hẳn trong việc loại bỏ nhiễu đầu vào mà không làm biến dạng dữ liệu gốc.
3. Exponential Smoothing (Làm mịn hàm mũ - Ví dụ 256): Phương pháp này gán trọng số cho các điểm dữ liệu, trong đó trọng số giảm theo hàm mũ đối với các điểm trong quá khứ. Điểm gần nhất có trọng số cao nhất, điểm càng xa trọng số càng thấp.	Cách hoạt động: Nó giống như một "bộ lọc có trí nhớ ngắn hạn". Hệ số α ($0 < \alpha < 1$) quyết định trí nhớ đó ngắn đến đâu. α càng lớn, mô hình càng tin vào dữ liệu mới nhất và quên đi quá khứ nhanh chóng. Kỹ thuật: Giá trị làm mịn S_t tại thời điểm t được tính bằng: $S_t = \alpha * Y_t + (1 - \alpha) * S_{t-1}$, trong đó α là hệ số làm mịn ($0 < \alpha < 1$), Y_t là giá trị thực tại t, và S_{t-1} là giá trị làm mịn tại $t-1$. Bối cảnh: Một cửa hàng bán kem muốn dự báo doanh số cho ngày mai để chuẩn bị nguyên liệu. Thực tiễn: Nếu thời tiết đột ngột nắng nóng, doanh số hôm qua sẽ tăng vọt. Với α lớn (ví dụ 0.7), dự báo cho ngày mai sẽ tăng mạnh theo $(0.7 * \text{Doanh_số_cao} + 0.3 * \text{Dự_báo_cũ})$, phản ánh kỳ vọng rằng thời tiết nắng nóng sẽ còn tiếp diễn. Điều này giúp chủ cửa hàng chuẩn bị đủ kem, tránh thất thoát doanh thu.	

2.2. Biến đổi và Rút trích Đặc trưng (Transformation & Feature Extraction)

2.2.1. Biến đổi (Transformation) là gì?

Đây là việc thay đổi hình thức biểu diễn của dữ liệu để giải quyết các vấn đề như tính không ổn định (non-stationary) hoặc để chuẩn bị cho các thuật toán cụ thể.

Trường hợp: Khi dữ liệu có xu hướng (trend) hoặc phương sai thay đổi (volatility clustering trong tài chính).

Cách làm: Phổ biến nhất là Lấy sai phân (Differencing) và Lấy logarit. Lấy sai phân để loại bỏ xu hướng, còn lấy logarit giúp ổn định phương sai và biến các quan hệ nhân tính thành quan hệ cộng tính, dễ mô hình hóa hơn.

2.2.2. Rút trích Đặc trưng (Feature Extraction) là gì?

Là quá trình tạo ra các đặc trưng mới, có ý nghĩa từ dữ liệu thô, thường với mục đích giảm chiều dữ liệu mà vẫn giữ lại thông tin quan trọng.

Các phương pháp này nhằm mục đích làm ổn định chuỗi (stationary - không còn xu hướng, phương sai không đổi) hoặc giảm kích thước dữ liệu trong khi vẫn giữ lại đặc trưng quan trọng. Đây là bước cực kỳ cần thiết vì hầu hết các mô hình thống kê và học máy đều giả định dữ liệu stationary. Ứng dụng thực tiễn bao gồm dự báo tài chính, dự báo nhu cầu, phát hiện bất thường trong sản xuất.

2.2.3. Differencing (Lấy sai phân):

Được dùng để loại bỏ xu hướng (trend) trong dữ liệu. Xu hướng mạnh có thể khiến mô hình đưa ra dự báo sai lệch. Một ví dụ kinh điển là mối tương quan giả giữa "số người chết đuối" và "doanh số bán kem" - cả hai đều có xu hướng tăng vào mùa hè. Nếu không loại bỏ xu hướng mùa, một mô hình ngây thơ có thể kết luận mua kem nhiều gây chết đuối! Lấy sai phân giúp loại bỏ yếu tố nhiễu này để tìm ra các mối quan hệ thực sự.

2.2.3.1. Sai phân là gì? Là chênh lệch giữa giá trị tại thời điểm t và thời điểm $t-k$.

2.2.3.1.1. **First-Order Differencing (Sai phân bậc 1 - Ví dụ 257): Tính chênh lệch giữa điểm liên kế ($t - (t-1)$).**

Công thức: $\text{diff}(t) = Y(t) - Y(t-1)$

Ví dụ: Nếu nhiệt độ các ngày là [28, 30, 29, 31] °C. Sai phân bậc 1 là [NaN, 2, -1, 2].

2.2.3.1.2. **Second-Order Differencing (Sai phân bậc 2 - Ví dụ 258): Lấy sai phân một lần nữa trên kết quả của sai phân bậc một, giúp loại bỏ xu hướng bậc cao hơn (ví dụ: xu hướng bậc 2 - parabolic).**

Công thức: $\text{diff2}(t) = \text{diff}(t) - \text{diff}(t-1) = [Y(t) - Y(t-1)] - [Y(t-1) - Y(t-2)] = Y(t) - 2*Y(t-1) + Y(t-2)$

Ví dụ: Từ kết quả sai phân bậc 1 [NaN, 2, -1, 2], sai phân bậc 2 là [NaN, NaN, -3, 3].

2.2.3.2. Piecewise Aggregate Approximation (PAA - Ví dụ 259):

Là gì? Chia chuỗi thời gian dài thành các đoạn (segment) có độ dài bằng nhau và lấy giá trị trung bình của mỗi đoạn để đại diện cho đoạn đó.

Mục đích/Tại sao cần? Nó giải quyết vấn đề kích thước dữ liệu quá lớn. Một chuỗi có 1000 điểm có thể được PAA thành 10 đoạn, mỗi đoạn 100 điểm, được đại diện bởi 10 giá trị trung bình. Điều này giúp giảm chi phí tính toán đáng kể cho các thuật toán so sánh hay khai phá chuỗi thời gian, trong khi vẫn giữ được hình dạng tổng quát của chuỗi.

Ví dụ: Nén dữ liệu điện năng tiêu thụ theo giờ trong 1 tháng (720 điểm) thành giá trị trung bình cho mỗi ngày (30 điểm), giúp việc vẽ biểu đồ và phân tích xu hướng hàng ngày dễ dàng hơn.

2.2.3.3. Autocorrelation (Tự tương quan - Ví dụ 260):

Giải thích: Đo lường mối tương quan của một chuỗi thời gian với chính nó tại các độ trễ (lag) khác nhau. Độ trễ k có nghĩa là so sánh chuỗi gốc với chính nó được dịch chuyển đi k bước thời gian.

Ví dụ về độ trễ: Với chuỗi doanh số [100, 150, 200, 250], lag=1 là [150, 200, 250]. Autocorrelation tại lag=1 sẽ đo xem [100,150,200] có tương quan với [150,200,250] không.

Đặc tính/Ứng dụng: Nó giúp phát hiện tính thời vụ (seasonality) và tính tuần hoàn. Ví dụ: Chuỗi doanh số bán hàng hàng ngày có thể có autocorrelation cao tại lag=7, lag=14,... chứng tỏ có tính thời vụ theo tuần.

2.2.3.4. Regression Line (Đường hồi quy - Ví dụ 261):

Giải thích: Sử dụng phương pháp Bình phương tối thiểu (Least Squares) để tìm một đường thẳng (hoặc đường cong) phù hợp nhất với dữ liệu, thể hiện xu hướng dài hạn (trend).

Bình phương tối thiểu là gì? Đây là phương pháp tối ưu hóa để tìm ra các tham số của đường hồi quy (ví dụ: hệ số góc và intercept) sao cho tổng bình phương của các khoảng cách (sai số) từ các điểm dữ liệu đến đường hồi quy là nhỏ nhất. Chính đặc tính tối ưu này giúp nó đưa ra được đường thể hiện xu hướng "tốt nhất" theo một tiêu chí toán học rõ ràng.

2.3. Phân đoạn và Biểu diễn Ký hiệu (Segmentation & Symbolic Representation)

a. Split Time Series (Ví dụ 262, 263):

Ứng dụng thực tiễn: Ngoài việc tạo dữ liệu huấn luyện, nó còn dùng để phân tích từng giai đoạn cụ thể (ví dụ: tách doanh số theo quý, theo năm để phân tích riêng biệt), hoặc chuẩn bị dữ liệu cho các mô hình yêu cầu đầu vào có độ dài cố định (như LSTM).

b. Symbolic Aggregate Approximation (SAX - Ví dụ 264):

Cách hoạt động: SAX thực hiện hai bước chính:

PAA: Đầu tiên, nó sử dụng PAA để giảm kích thước chuỗi thời gian, thu được một chuỗi các giá trị trung bình.

Chuyển đổi sang ký hiệu: Sau đó, nó ánh xạ các giá trị trung bình này vào các ký tự dựa trên các ngưỡng được xác định từ phân phối chuẩn (Gaussian distribution). Ví dụ: Nếu chia thành 3 ký tự a, b, c, các giá trị PAA nằm trong vùng thấp nhất sẽ thành a, vùng trung bình thành b, vùng cao nhất thành c.

Đóng góp/Ứng dụng thực tiễn: Ưu điểm lớn nhất của SAX là biến đổi dữ liệu số phức tạp thành chuỗi ký tự đơn giản. Điều này cho phép áp dụng các thuật toán Khai phá mẫu trình tự (Sequential Pattern Mining) và Luật kết hợp (Association Rules) cực kỳ mạnh mẽ lên dữ liệu chuỗi thời gian.

Ví dụ 1: Một cảm biến nhiệt độ ghi nhận dữ liệu mỗi phút được chuyển thành chuỗi ký tự SAX: abbcccdcbba. Thuật toán có thể phát hiện ra rằng mẫu hình "ccc" (nhiệt độ cao kéo dài 3 phút) thường xuyên xuất hiện trước khi máy nén trong hệ thống lạnh bật lên. Đây chính là một "luật kết hợp" có ý nghĩa trong việc bảo trì dự đoán.

Ví dụ 2: Bạn có dữ liệu điện năng tiêu thụ theo giờ của một hộ gia đình trong 1 tháng (30 ngày * 24 giờ = 720 điểm). Bạn muốn phân tích xu hướng tiêu thụ theo ngày.

Áp dụng PAA: Chia chuỗi 720 điểm thành 30 đoạn (ứng với 30 ngày), mỗi đoạn dài 24 giờ. Tính trung bình điện năng tiêu thụ cho mỗi ngày.

Kết quả: Bạn thu được một chuỗi mới chỉ có 30 điểm, mỗi điểm đại diện cho mức tiêu thụ trung bình một ngày. Chuỗi này về ra sẽ cho bạn thấy xu hướng tiêu thụ điện tăng/giảm qua các ngày trong tháng một cách rõ ràng và dễ quan sát hơn nhiều so với nhìn vào 720 điểm gốc.